

المرجع الرسمي للغة البرمجة UNL

Unteck

دليل تعليمي شامل من الأساسيات إلى الاحتراف

هيم خليل

حقوق النشر

اسم الكتاب: المرجع الرسمي للغة البرمجة UNL

الإصدار: الإصدار الأول

لغة البرمجة: UNL (Unteck Normal Language)

المؤلف: خالف إبراهيم خليل

© جميع الحقوق محفوظة للمؤلف.

لا يجوز إعادة نشر هذا الكتاب أو نسخه أو توزيعه أو ترجمته أي جزء منه بأي وسيلة كانت، سواء إلكترونية أو مطبوعة، دون الحصول على إذن كتابي مسبق من المؤلف، باستثناء الاقتباسات القصيرة المستخدمة لأغراض التعليم أو المراجعة مع الإشارة إلى المصدر.

جميع أسماء المشاريع والمنتجات المذكورة في هذا الكتاب تعود إلى أصحابها إذا كانت علامات تجارية مسجلة.

تم إعداد هذا الكتاب ليكون المرجع الرسمي لتعلم لغة البرمجة UNL، وقد تمت كتابة جميع الأمثلة والشروحات لأغراض تعليمية.

قد تتغير بعض خصائص اللغة أو بيئة التطوير في الإصدارات المستقبلية، لذلك ينصح دائمًا بالرجوع إلى أحدث الوثائق الرسمية عند توفرها.

طبع هذا الكتاب للمرة الأولى في الإصدار الأول من لغة البرمجة UNL.

كلمة المؤلف

بسم الله الرحمن الرحيم

الحمد لله الذي علّم بالقلم، علّم الإنسان ما لم يعلم.

منذ بداية رحلتي مع البرمجة، كان يراودني حلم إنشاء لغة برمجة خاصة، لغة تجمع بين سهولة التعلم وقوة الإمكانيات، وتمنح المطور بيئة موحدة لبناء مختلف أنواع المشاريع دون الحاجة إلى الانتقال المستمر بين لغات متعددة.

بدأ هذا المشروع كمجرد فكرة، ثم تحول إلى ساعات طويلة من التعلم والتجربة والتطوير، حتى أصبحت تلك الفكرة مشروعًا حقيقيًا يحمل اسم UNL.

لم يكن الهدف إنشاء لغة جديدة لمجرد إضافة اسم آخر إلى عالم البرمجة، بل كان الهدف تصميم لغة واضحة، بسيطة، وعملية، يمكن استخدامها في تطوير تطبيقات الويب، والخوادم، والهواتف الذكية، والذكاء الاصطناعي، وتحليل البيانات، والأمن السيبراني، وإنترنت الأشياء، وغيرها من المجالات الحديثة.

هذا الكتاب ليس مجرد شرح لأوامر اللغة، بل هو دليل يساعد على فهم طريقة التفكير التي بُنيت عليها UNL، وكيفية استخدامها لبناء مشاريع حقيقية بطريقة منظمة واحترافية.

أتمنى أن يكون هذا الكتاب بداية موفقة لكل من يرغب في تعلم لغة UNL، وأن يجد فيه القارئ ما يساعده على تطوير مهاراته وتحويل أفكاره إلى برامج ومشاريع واقعية.

وفي الختام، أسأل الله أن ينفع بهذا العمل، وأن يكتب له القبول، وأن يكون بداية لمستقبل مشرق لهذه اللغة ولكل من يستخدمها.

خالف إبراهيم خليل

مؤسس ومطور لغة البرمجة UNL

الجزء الأول: البداية مع UNL

1. مقدمة عن لغة UNL
2. فلسفة لغة UNL
3. لماذا تم إنشاء UNL؟
4. مجالات استخدام UNL
5. مميزات لغة UNL
6. تثبيت بيئة التطوير UNL Studio
7. كتابة أول برنامج

الجزء الثاني: أساسيات اللغة

1. المتغيرات
2. أنواع البيانات
3. الثوابت
4. النصوص
5. العمليات الحسابية
6. العمليات المنطقية
7. الشروط
8. الحلقات
9. الدوال
10. الإرجاع (Return)
11. الكائنات (Objects)
12. المجموعات (Sets)
13. المصفوفات (Arrays)
14. معالجة الأخطاء
15. البرمجة غير المتزامنة (Async)

الجزء الثالث: المكتبة القياسية

1. الوسيط un.math
2. الوسيط un.web

- 3. نظام Style
- 4. الوسيط un.gui
- 5. الوسيط un.back
- 6. الوسيط un.net
- 7. الوسيط un.ai
- 8. الوسيط un.cyber
- 9. الوسيط un.data
- 10. الوسيط un.iot
- 11. الوسيط un.native
- 12. الوسيط un.apk
- 13. الوسيط un.ios

الجزء الرابع: تنظيم المشاريع

- 1. Link.file
- 2. الوحدات (Modules)
- 3. Package Manager

الجزء الخامس: كيف تعمل لغة UNL؟

- 1. ما هو Compiler؟
- 2. Lexer
- 3. Parser
- 4. AST
- 5. Semantic Analyzer
- 6. IR (Intermediate Representation)
- 7. LLVM
- 8. Machine Code

الجزء السادس: UNL Studio

- 1. التعرف على بيئة التطوير
- 2. إنشاء مشروع جديد
- 3. محرر الأكواد
- 4. المستكشف (Explorer)

- 5. الطرفية (Console)
- 6. المترجم (Compiler)
- 7. مساعد الذكاء الاصطناعي
- 8. بناء التطبيقات

الجزء السابع: المشاريع العملية

- 1. مشروع آلة حاسبة
- 2. مشروع تطبيق ملاحظات
- 3. مشروع موقع ويب
- 4. مشروع واجهة رسومية
- 5. مشروع API
- 6. مشروع نظام تسجيل دخول آمن
- 7. مشروع مساعد ذكاء اصطناعي
- 8. مشروع إنترنت الأشياء
- 9. مشروع متكامل باستخدام UNL

الجزء الثامن: الاحتراف

- 1. أفضل الممارسات
- 2. تنظيم المشاريع الكبيرة
- 3. تحسين الأداء
- 4. الأخطاء الشائعة وكيفية تجنبها
- 5. نصائح للمطورين

الخاتمة

- 1. رسالة إلى القارئ
- 2. مستقبل لغة UNL
- 3. كلمة الختام

مقدمة عن لغة UNL

"وراء كل لغة برمجة فكرة، ووراء كل فكرة رحلة من التعلم والإبداع"

تُعد **UNL (Unteck Normal Language)** لغة برمجة حديثة متعددة الأغراض، صُممت لتوفير تجربة تطوير موحدة تجمع بين البساطة والمرونة والقوة. وتهدف إلى تمكين المطور من بناء أنواع مختلفة من البرمجيات باستخدام لغة واحدة، مع الحفاظ على أسلوب كتابة واضح وسهل القراءة.

بدأ مشروع **UNL** كمشروع فردي أنشأه **خالف إبراهيم خليل**، حيث بدأ تطوير اللغة في سن الرابعة عشرة، واستمر العمل عليها لمدة ثلاثة أشهر متواصلة، بهدف إنشاء لغة برمجة حديثة تناسب احتياجات المطورين في مختلف المجالات.

لا تقتصر UNL على مجال واحد، بل صُممت لتكون لغة عامة يمكن استخدامها في تطوير مواقع الويب، والخوادم، وتطبيقات الهواتف الذكية، والذكاء الاصطناعي، وتحليل البيانات، والأمن السيبراني، وإنترنت الأشياء، وبرامج سطح المكتب، وغيرها من المجالات البرمجية.

ومنذ بداية تصميمها، كان الهدف الأساسي هو تقليل التعقيد الموجود في العديد من اللغات الحديثة، مع المحافظة على القدرة على إنشاء مشاريع كبيرة ومنظمة دون التضحية بسهولة التعلم.

يعتمد تصميم اللغة على قواعد واضحة وأسلوب كتابة بسيط، بحيث يستطيع المبتدئ تعلمها بسرعة، بينما يجد المطور المحترف الأدوات التي يحتاجها لبناء تطبيقات متقدمة.

ولا يقتصر مشروع **UNL** على اللغة فقط، بل يشمل منظومة تطوير كاملة تتكون من:

- لغة البرمجة UNL.
 - بيئة التطوير الرسمية **UNL Studio**.
 - مكتبة قياسية (Standard Library).
 - أدوات لبناء التطبيقات.
 - نظام لتنظيم المشاريع.
 - مترجم (Compiler) حديث يعتمد على مراحل متعددة لتحويل الشيفرة المصدرية إلى برنامج قابل للتنفيذ.
- سيأخذك هذا الكتاب في رحلة تبدأ من أساسيات اللغة، ثم تنتقل تدريجيًا إلى المفاهيم المتقدمة، مع الكثير من الأمثلة العملية والمشاريع الواقعية التي تساعد على فهم اللغة بصورة صحيحة.
- إذا كانت هذه أول مرة تتعرف فيها على UNL، فهذا الكتاب كُتب خصيصًا ليكون نقطة البداية المناسبة، أما إذا كانت لديك خبرة سابقة في البرمجة، فستجد فيه مرجعًا متكاملًا يساعدك على إتقان اللغة والاستفادة من جميع إمكانياتها.
- مرحبًا بك في عالم **UNL**، ونتمنى أن تكون هذه الرحلة بداية لمشاريع مميزة وأفكار تتحول إلى تطبيقات حقيقية.

الفصل الثاني

فلسفة لغة UNL

لكل لغة برمجة فلسفة خاصة تحدد طريقة تصميمها، وأسلوب كتابة الشيفرة، والمشكلات التي تحاول حلها. أما **UNL** فقد بُنيت على مجموعة من المبادئ التي تهدف إلى جعل البرمجة أكثر وضوحًا وسهولة، دون التضحية بالإمكانات التي يحتاجها المطور في المشاريع الكبيرة.

لا تهدف UNL إلى أن تكون مجرد لغة جديدة، بل إلى أن تكون بيئة تطوير متكاملة تساعد المطور على إنجاز مختلف أنواع المشاريع باستخدام لغة واحدة، مع تقليل الحاجة إلى تعلم لغات متعددة لكل مجال.

البساطة أولاً

أحد أهم مبادئ UNL هو أن تكون الشيفرة سهلة القراءة والفهم حتى بعد مرور فترة طويلة على كتابتها.

فكلما كانت الشيفرة أوضح، أصبح تطوير المشروع وصيانتها أسهل، وانخفضت نسبة الأخطاء البرمجية.

ولهذا ضُممت معظم أوامر اللغة لتكون قريبة من المعنى الذي تؤديه، بحيث يستطيع المطور فهم وظيفة الكود بسرعة.

لغة واحدة لمجالات متعددة

في كثير من المشاريع يضطر المطور إلى تعلم عدة لغات وتقنيات مختلفة لإنجاز تطبيق واحد.

أما في UNL فالهدف هو تقليل هذا التعقيد، وذلك بتوفير وسطاء (Mediators) متخصصة لكل مجال، مثل:

▪ **un.web** لتطوير الويب.

▪ **un.back** لتطوير الخوادم.

▪ **un.gui** لتطبيقات سطح المكتب.

▪ **un.apk** و **un.ios** لتطبيقات الهواتف.

▪ **un.ai** للذكاء الاصطناعي.

▪ **un.cyber** للأمن السيبراني.

▪ **un.data** لتحليل البيانات.

▪ **un.net** للشبكات وواجهات API.

▪ **un.iot** لإنترنت الأشياء.

▪ **un.math** للحوسبة الرياضية.

▪ **un.native** لبرمجة الأنظمة.

وبذلك يستطيع المطور الانتقال بين مجالات البرمجة المختلفة مع الاحتفاظ بنفس أسلوب الكتابة.

سهولة التعلم

تم تصميم قواعد اللغة بحيث تكون مباشرة، مع تقليل الرموز غير الضرورية. كما أن اللغة غير حساسة لحالة الأحرف (Case Insensitive)، مما يمنح المطور مرونة أكبر أثناء كتابة الشيفرة.

تنظيم المشاريع

لا تعتمد المشاريع الكبيرة على كتابة الكود فقط، بل تعتمد أيضًا على حسن تنظيم الملفات وتقسيمها. لذلك تدعم UNL تقسيم المشروع إلى وحدات مستقلة وربطها بسهولة، مما يجعل المشاريع أكثر ترتيبًا وأسهل في الصيانة والتطوير.

قابلية التوسع

قد يبدأ المشروع صغيرًا، ثم يتحول لاحقًا إلى نظام ضخم. لهذا صُممت UNL بحيث تسمح بإضافة ملفات جديدة، ووحدات إضافية، ومكتبات، دون الحاجة إلى إعادة بناء المشروع من الصفر.

الأداء

رغم أن اللغة تركز على سهولة الاستخدام، فإنها لا تهمل الأداء. ويعتمد مترجم UNL على سلسلة من المراحل تبدأ بتحليل الشيفرة، ثم بناء شجرة البرنامج، ثم إنشاء تمثيل بسيط، وصولًا إلى إنتاج كود تنفيذي عالي الكفاءة. وهذا يهدف إلى الجمع بين سهولة التطوير وسرعة التنفيذ.

مستقبل اللغة

تم تصميم UNL لتكون لغة قابلة للتطور مع مرور الوقت.

ولهذا يمكن إضافة وسطاء جديدة، وتطوير المكتبة القياسية، وتحسين بيئة التطوير، دون التأثير على المشاريع المكتوبة بالإصدارات السابقة.

إن فلسفة UNL تقوم على فكرة بسيطة:

لغة واحدة، أسلوب واحد، وإمكانات واسعة لبناء مختلف أنواع البرمجيات.

وهذا المبدأ هو الأساس الذي بُنيت عليه جميع مكونات اللغة، وهو ما ستلاحظه في بقية فصول هذا الكتاب.

الفصل الثالث

لماذا تم إنشاء UNL؟

تطورت لغات البرمجة كثيرًا خلال العقود الماضية، وأصبحت كل لغة تتميز بمجال معين أو تقدم حلولًا لمشكلة محددة. ومع ذلك، ما زال المطور يواجه تحديًا متكررًا، وهو الحاجة إلى تعلم عدة لغات وأدوات مختلفة لإنجاز مشروع واحد.

فقد يحتاج المطور إلى لغة لتطوير واجهات الويب، ولغة أخرى للخوادم، وأخرى لتطبيقات الهواتف، وأخرى لتحليل البيانات أو الذكاء الاصطناعي، بالإضافة إلى تعلم مكتبات وأطر عمل مختلفة لكل مجال.

جاءت فكرة UNL لتقليل هذا التعقيد، وذلك من خلال إنشاء لغة برمجة حديثة يمكن استخدامها في مجالات متعددة، مع المحافظة على أسلوب كتابة موحد وسهل الفهم.

لم يكن الهدف منافسة جميع اللغات الموجودة، بل تقديم تجربة مختلفة تركز على جمع الأدوات الأساسية داخل منظومة واحدة، بحيث يقضي المطور وقتًا أقل في تعلم التقنيات المختلفة، ووقتًا أكبر في بناء المشاريع.

المشكلات التي تحاول UNL حلها

تسعى UNL إلى معالجة مجموعة من التحديات التي يواجهها المطورون، من أهمها:

أولاً: كثرة اللغات في المشروع الواحد

في كثير من المشاريع يضطر المطور إلى استخدام عدة لغات في الوقت نفسه.

أما في UNL، فالهدف هو تقليل هذا العدد قدر الإمكان، من خلال توفير وسطاء متخصصة تغطي مجالات متعددة داخل اللغة نفسها.

ثانيًا: صعوبة الانتقال بين المجالات

قد يرغب المطور في الانتقال من تطوير الويب إلى الذكاء الاصطناعي أو إلى برمجة الأنظمة، لكنه يجد نفسه مضطرًا لتعلم لغة جديدة بالكامل.

في UNL تبقى طريقة الكتابة متقاربة مهما اختلف المجال، مما يجعل الانتقال أكثر سهولة.

ثالثًا: كثرة الأدوات الخارجية

تعتمد بعض اللغات على عدد كبير من المكتبات والإعدادات قبل البدء في العمل.

أما UNL فتهدف إلى توفير عدد كبير من الإمكانيات بشكل منظم داخل مكتبتها القياسية ووسطائها المختلفة، لتقليل الوقت اللازم لإعداد المشروع.

رابعًا: سهولة القراءة

تم تصميم قواعد اللغة لتكون واضحة ومنظمة، حتى يتمكن المطور من فهم الشيفرة بسرعة، سواء كتبها بنفسه أو قرأها بعد فترة طويلة.

خامسًا: بناء مشاريع متكاملة

لا تقتصر UNL على البرامج الصغيرة، بل ضمنت أيضًا لتناسب المشاريع الكبيرة من خلال دعم تقسيم الملفات، والدوال، والكائنات، ومعالجة الأخطاء، والبرمجة غير المتزامنة، وقواعد البيانات، والشبكات، وغيرها من المكونات الضرورية.

رؤية المشروع

تتمثل رؤية UNL في إنشاء منظومة تطوير متكاملة تشمل:

- لغة برمجة حديثة.
- بيئة تطوير رسمية (UNL Studio).
- مترجم احترافي.
- مكتبة قياسية شاملة.
- مدير حزم.
- أدوات لبناء تطبيقات الويب.
- أدوات لتطوير تطبيقات الهواتف.
- أدوات لسطح المكتب.
- أدوات للذكاء الاصطناعي.
- أدوات للأمن السيبراني.
- أدوات لإنترنت الأشياء.
- أدوات لتحليل البيانات.

الهدف النهائي

الهدف من UNL ليس مجرد كتابة الشيفرة، بل بناء بيئة تجعل عملية تطوير البرمجيات أكثر بساطة وتنظيمًا، مع توفير أدوات تساعد المطور على تحويل فكرته إلى مشروع حقيقي بأقل قدر ممكن من التعقيد.

وقد بدأت UNL كمشروع فردي، لكنها ضُمت منذ البداية برؤية تسمح لها بالنمو والتطور مستقبلاً، لتصبح منصة متكاملة يمكن الاعتماد عليها في مختلف مجالات البرمجة.

بهذا تنتهي مقدمة الكتاب، ومن الفصل القادم سنبدأ العمل الفعلي مع اللغة، ابتداءً من **تثبيت بيئة التطوير UNL Studio**، ثم كتابة أول برنامج بلغة UNL.

مقدمة عن لغة UNL

تُعد **UNL (Unteck Normal Language)** لغة برمجة حديثة صُممت لتجمع بين سهولة التعلم وقوة الأداء، مع هدف واضح يتمثل في توفير لغة واحدة يمكن استخدامها في معظم مجالات البرمجة الحديثة.

بدأ تطوير اللغة كمشروع فردي على يد **خالف إبراهيم خليل**، مدفوعًا بالرغبة في إنشاء لغة توفر بيئة موحدة للمبرمج، دون الحاجة إلى الانتقال المستمر بين لغات مختلفة عند بناء المشاريع.

تدعم لغة **UNL** العديد من المجالات، منها:

- تطوير تطبيقات الويب (Front-end).
- تطوير الخوادم (Back-end).
- تطوير تطبيقات iOS و APK.
- الذكاء الاصطناعي وتعلم الآلة.
- الأمن السيبراني.
- تحليل البيانات وعلوم البيانات.
- برمجة الأنظمة.
- تطبيقات سطح المكتب.
- إنترنت الأشياء (IoT).
- الشبكات وواجهات البرمجة (APIs).
- البرامج النصية (Console).

تم تصميم بناء اللغة ليكون بسيطًا وواضحًا، بحيث يستطيع المبتدئ تعلمها بسهولة، وفي الوقت نفسه توفر إمكانيات كافية لبناء المشاريع الاحترافية.

يعتمد تصميم UNL على فكرة دمج العديد من المجالات داخل لغة واحدة باستخدام وسائط (Modules) متخصصة، مثل **un.web** و **un.ai** و **un.net** وغيرها، مما يجعل الانتقال بين المجالات أكثر تنظيمًا.

هذا الكتاب سيأخذك خطوة بخطوة، بدءًا من كتابة أول برنامج، وحتى بناء تطبيقات متكاملة باستخدام جميع إمكانيات لغة UNL.

الفصل الخامس

فلسفة اللغة

لا تقوم فلسفة UNL على زيادة عدد الأوامر أو تعقيد بناء الجمل، بل تعتمد على مبدأ بسيط:

لغة واحدة... لكل مشروع تقريبًا.

فبدل تعلم عدة لغات لكل مجال، تسعى UNL إلى توفير تجربة موحدة، مع الحفاظ على سهولة القراءة وبساطة كتابة الكود.

ترتكز فلسفة اللغة على عدة مبادئ:

- سهولة التعلم.
- سرعة كتابة البرامج.
- تنظيم المشاريع.
- قابلية التوسع.
- دعم مجالات متعددة.
- تقليل التعقيد.
- توفير تجربة تطوير موحدة.

كما تعتمد اللغة على تقسيم الوظائف إلى وسائط متخصصة، مما يجعل الكود أكثر ترتيبًا ووضوحًا.

الفصل السادس

لماذا تم إنشاء UNL؟

ظهرت فكرة UNL نتيجة ملاحظة أن المبرمج غالبًا يحتاج إلى تعلم عدد كبير من اللغات المختلفة لإنجاز مشروع واحد. فعلى سبيل المثال، قد يحتاج المشروع إلى لغة للواجهة الأمامية، وأخرى للخادم، وثالثة لتطبيق الهاتف، ورابعة للذكاء الاصطناعي.

جاءت UNL لتقليل هذا التعقيد من خلال محاولة جمع هذه المجالات داخل لغة واحدة، مع الحفاظ على بساطة الكتابة.

ومن أهم أهداف المشروع:

- تقليل عدد اللغات المطلوبة.
- تسهيل تعلم البرمجة.
- تسريع تطوير المشاريع.
- توفير مكتبات موحدة.
- إنشاء بيئة تطوير رسمية.
- دعم بناء تطبيقات متعددة المنصات.

إن UNL ليست مجرد لغة برمجة، بل مشروع يهدف إلى إنشاء منظومة تطوير متكاملة تشمل اللغة، والمترجم، وبيئة التطوير، والمكتبات القياسية.

تثبيت بيئة التطوير UNL Studio

الآن نحن سوف نثبت المحرر الخاص باللغة اولاً نذهب الى الرابط التالي

[/https://brahimkha959-spec.github.io/Unl-studio](https://brahimkha959-spec.github.io/Unl-studio)

هناك سوف تجد بعض الاشياء عن اللغة وايضا سوف تجد اماكن التحميل حمل حمل نسخته المحرر حسب الجهاز الذي لديك
عندما تحمله ننتقل الى الفصل التالي

الفصل الثامن

كتابة أول برنامج

بعد تثبيت **UNL Studio**، أصبح كل شيء جاهزًا لكتابة أول برنامج بلغة UNL.

يُعد برنامج "مرحبًا بالعالم" أول خطوة في تعلم أي لغة برمجة، لأنه يوضح البنية الأساسية للبرنامج.

مثال:

```
Print("Welcome in UNL")

Unteck run.u(bg)
```

في هذا المثال:

- تستخدم الدالة **Print()** لطباعة النص.
- توضع النصوص بين علامات الاقتباس.
- ينتهي البرنامج دائمًا بالسطر:

```
Unteck run.u(bg)
```

وهو السطر الذي يشير إلى نهاية البرنامج.

يمكن تجربة تعديل النص داخل الدالة **Print** وتشغيل البرنامج عدة مرات للتعرف على طريقة عمل اللغة.

بعد فهم هذا المثال، يصبح المستخدم مستعدًا للانتقال إلى تعلم المتغيرات وأنواع البيانات وبقية أساسيات لغة UNL.

الفصل التاسع

العمليات الحسابية

تدعم لغة **UNL** مجموعة متكاملة من العمليات الحسابية التي يمكن استخدامها مع الأعداد والمتغيرات ونتائج الدوال.

تُستخدم هذه العمليات في بناء البرامج العلمية، والتجارية، والألعاب، والتطبيقات المختلفة.

عملية الجمع (+)

تجمع قيمتين أو أكثر.

مثال:

```
a = 10  
b = 5  
  
Print(a + b)  
  
Unteck run.u(bg)
```

النتيجة:

15

عملية الطرح (-)

تطرح قيمة من أخرى.

```
a = 20  
b = 8  
  
Print(a - b)  
  
Unteck run.u(bg)
```

النتائج:

12

عملية الضرب (*)

```
a = 6  
b = 7  
  
Print(a * b)  
  
Untrack run.u(bg)
```

النتائج:

42

عملية القسمة (/)

```
a = 30  
b = 5  
  
Print(a / b)  
  
Untrack run.u(bg)
```

النتائج:

6

باقي القسمة (%)

تعيد باقي قسمة عدد على آخر.

```
Print(10 % 3)

Unteck run.u(bg)
```

النتائج:

1

الأس (^)

لحساب القوة.

```
Print(2 ^ 5)

Unteck run.u(bg)
```

النتائج:

32

الجذر (√)

لحساب الجذر التربيعي.

```
Print(√25)

Unteck run.u(bg)
```

النتائج:

العمليات باستخدام المتغيرات

```
price = 150
tax = 25

total = price + tax

Print(total)

Unteck run.u(bg)
```

ترتيب العمليات

تنفذ UNL العمليات الحسابية حسب الأولوية الرياضية المعتادة.

مثال:

```
Print(5 + 3 * 2)

Unteck run.u(bg)
```

النتاج:

11

أما باستخدام الأقواس:

```
Print((5 + 3) * 2)

Unteck run.u(bg)
```

النتاج:

16

مثال عملي: حساب مساحة مستطيل

```
width = 20
height = 15

area = width * height

Print(area)

Unteck run.u(bg)
```

مثال عملي: حساب متوسط الدرجات

```
math = 18
physics = 16
english = 19

average = (math + physics + english) / 3

Print(average)

Unteck run.u(bg)
```

تُعد العمليات الحسابية من أكثر الأدوات استخدامًا داخل لغة **UNL**، وستجدها في جميع أنواع المشاريع، سواء كانت تطبيقات، أو ألعابًا، أو أنظمة مالية، أو برامج علمية.

في الفصل القادم سنتعرف على **الشروط (Conditions)**، وكيفية اتخاذ القرارات داخل البرنامج باستخدام **If** و **Elif** و **Else**.

الفصل العاشر

الشروط (Conditions)

لا تعتمد البرامج على تنفيذ الأوامر بالترتيب فقط، بل تحتاج أحيانًا إلى اتخاذ قرارات بناءً على قيم معينة. ولهذا توفر لغة UNL نظامًا متكاملًا للشروط باستخدام **If** و **Elif** و **Else**.

يسمح هذا النظام بتنفيذ جزء معين من الشيفرة إذا تحقق شرط معين، أو تنفيذ جزء آخر إذا لم يتحقق.

جملة If

تُستخدم لتنفيذ الشيفرة عندما يكون الشرط صحيحًا.

مثال:

```
age = 20

If(age ≥ 18)={
    Print("مسموح بالدخول")
}:

Unteck run.u(bg)
```

النتائج:

مسموح بالدخول

جملة Else

إذا لم يتحقق الشرط، يتم تنفيذ الكود الموجود داخل Else.

مثال:


```

age = 15

If(age ≥ 18)={
    Print("بالغ")
}:
Else={
    Print("قاصر")
}:

Unteck run.u(bg)

```

جملة Elif

تُستخدم عندما توجد عدة احتمالات.

مثال:

```

score = 82

If(score ≥ 90)={
    Print("ممتاز")
}:
Elif(score ≥ 75)={
    Print("جيد جداً")
}:
Elif(score ≥ 50)={
    Print("ناجح")
}:
Else={
    Print("راسب")
}:

Unteck run.u(bg)

```

المقارنة بين القيم

تدعم UNL جميع عمليات المقارنة الأساسية.

المعنى	العملية
يساوي	==

العملية	المعنى
=!	لا يساوي
<	أكبر من
>	أصغر من
=<	أكبر أو يساوي
=>	أصغر أو يساوي

مثال:

```
number = 50

If(number = 50)={
    Print("تم العثور على الرقم")
}:

Unteck run.u(bg)
```

العمليات المنطقية

يمكن دمج أكثر من شرط داخل جملة واحدة.

AND

يجب أن يتحقق الشرطان.

```
age = 25

If(age ≥ 18 && age ≤ 60)={
    Print("مسموح")
}:

Unteck run.u(bg)
```

OR

يكفي تحقق أحد الشرطين.

```
role = "Admin"

If(role = "Admin" || role = "Owner")={
    Print("لديك صلاحية كاملة")
}:

Unteck run.u(bg)
```

NOT

لعكس نتيجة الشرط.

```
online = false

If(!online)={
    Print("المستخدم غير متصل")
}:

Unteck run.u(bg)
```

استخدام الثوابت داخل الشروط

يمكن مقارنة المتغيرات مع الثوابت.

```
value = pi$:

If(value = pi:$:)= {
    Print("π القيمة هي")
}:

Unteck run.u(bg)
```

مثال عملي

```
username = "Ali"
password = "1234"

If(username = "Ali" && password = "1234")={
    Print("تم تسجيل الدخول")
}:
Else={
    Print("بيانات غير صحيحة")
}:

Unteck run.u(bg)
```

تُستخدم الشروط في جميع المشاريع تقريبًا، مثل تسجيل الدخول، والتحقق من البيانات، وإدارة الصلاحيات، واتخاذ القرارات داخل البرامج.

الفصل الحادي عشر

الحلقات (Loops)

تُستخدم الحلقات لتكرار تنفيذ مجموعة من الأوامر عدة مرات دون الحاجة إلى إعادة كتابة الشيفرة.

توفر لغة **UNL** أكثر من طريقة للتكرار، مما يجعلها مناسبة لمختلف أنواع البيانات.

While.t

تستمر الحلقة ما دام الشرط صحيحًا.

```
counter = 1
While.t(loop)+[
    Print(counter)
    counter = counter + 1
    Break(loop,5)
]:
Unteck run.u(bg)
```

While.f

تعمل حتى يصبح الشرط غير محقق أو يتم إيقافها.

```
While.f(loop)+[
    Print("Running")
    Break(loop,1)
]:
Unteck run.u(bg)
```

Break

تستخدم لإيقاف الحلقة.

```
While.t(numbers)+[
    Print("Hello")
    Break(numbers,3)
]:
Unteck run.u(bg)
```

حلقة For

تستخدم للتكرار على المجموعات.

```
numbers = <1 2 3 4 5>:
for item in numbers ={
    Print(item)
}:
Unteck run.u(bg)
```

التكرار على النصوص

```
word = "UNL"

for char in word = {
    Print(char)
}:

Unteck run.u(bg)
```

النتائج:

```
U
N
L
```

التكرار على الكائنات

```
user = {
    name = "Ali"
    age = 20
}:

for key in user = {
    Print(key)
}:

Unteck run.u(bg)
```

التكرار على الأسماء

```
names = <"Ali" "Sara" "Omar">:

for name in names ={

    Print(name)

}:

Unteck run.u(bg)
```

مثال عملي

```
scores = <95 81 60 40>:

for score in scores={

    If(score ≥ 50)={
        Print("ناجح")
    }:
    Else={
        Print("راسب")
    }:

}:

Unteck run.u(bg)
```

تُستخدم الحلقات في قراءة الملفات، وتحليل البيانات، وإنشاء الألعاب، والتعامل مع قواعد البيانات، وغيرها من التطبيقات العملية.

الفصل الثاني عشر

الدوال (Functions)

الدوال هي مجموعة من الأوامر تُكتب مرة واحدة، ثم يمكن استدعاؤها في أي مكان داخل البرنامج. يساعد استخدام الدوال على تنظيم الشيفرة، وتقليل التكرار، وجعل البرامج أكثر سهولة في الصيانة.

إنشاء دالة

```
func welcome()={  
    Print("Welcome to UNL")  
}:  
  
welcome()  
  
Unteck run.u(bg)
```

الدوال مع المعاملات

```
func hello(name)={  
    Print(name)  
}:  
  
hello("Ali")  
  
Unteck run.u(bg)
```

إرجاع قيمة

```
func area(width,height)={  
    return width * height  
}:  
  
result = area(20,10)  
  
Print(result)  
  
Unteck run.u(bg)
```

أكثر من معامل

```
func sum(a,b)={  
    return a+b  
}:  
  
Print(sum(10,30))  
  
Unteck run.u(bg)
```

القيم الافتراضية

```
func hello(name="Guest")={  
    return "Hello " + name  
}:  
  
Print(hello())  
  
Print(hello("Ali"))  
  
Unteck run.u(bg)
```

الاستدعاء المتداخل

```
func double(x)={  
    return x*2  
}:  
  
func calc(y)={  
    return double(y)  
}:  
  
Print(calc(10))  
  
Unteck run.u(bg)
```

الاستدعاء الذاتي

```
func factorial(n)={  
  
    If(n≤1)={  
        return 1  
    }:  
  
    return n*factorial(n-1)  
}:  
  
Print(factorial(5))  
  
Unteck run.u(bg)
```

الدوال بدون Return

```
func welcome()={  
  
    Print("Hello")  
}:  
  
welcome()  
  
Unteck run.u(bg)
```

مثال عملي

```
func average(a,b,c)={  
    return (a+b+c)/3  
}:  
  
result = average(15,18,20)  
  
Print(result)  
  
Unteck run.u(bg)
```

تُعد الدوال من أهم الأدوات في لغة **UNL**، فهي تجعل البرامج أكثر تنظيماً، وتسمح بإعادة استخدام الشيفرة في أكثر من مكان، كما تُستخدم في جميع المشاريع الكبيرة تقريباً.

الفصل الثالث عشر

الكائنات (Objects)

تُستخدم الكائنات (Objects) لتنظيم البيانات داخل كيان واحد، مما يجعل التعامل مع المشاريع الكبيرة أكثر سهولة وتنظيماً. بدلاً من إنشاء عشرات المتغيرات المنفصلة، يمكن جمع جميع البيانات المتعلقة بشخص أو منتج أو تطبيق داخل كائن واحد.

إنشاء كائن

```
user = {  
    name = "Ali"  
    age = 20  
    country = "Algeria"  
}:  
  
Print(user.name)  
  
Untrack run.u(bg)
```

النتيجة:

```
Ali
```

الوصول إلى الخصائص

يمكن الوصول إلى أي خاصية باستخدام النقطة (.).

```
Print(user.age)

Print(user.country)

Unteck run.u(bg)
```

تعديل قيمة

يمكن تغيير قيمة أي خاصية.

```
user.age = 25

Print(user.age)

Unteck run.u(bg)
```

إضافة خاصية جديدة

```
user.job = "Developer"

Print(user.job)

Unteck run.u(bg)
```

إنشاء دالة داخل الكائن

```
func user.show()={

    Print(user.name)

}:

user.show()

Unteck run.u(bg)
```

كائنات متعددة

```
book = {  
    title = "UNL"  
    pages = 500  
}:  
Print(book.title)  
Unteck run.u(bg)
```

مثال عملي

```
student = {  
    name = "Ahmed"  
    score = 95  
}:  
If(student.score ≥ 50)={  
    Print(student.name)  
}:  
Unteck run.u(bg)
```

تُستخدم الكائنات في التطبيقات، والألعاب، وقواعد البيانات، والذكاء الاصطناعي، وجميع المشاريع الكبيرة تقريبًا.

الفصل الرابع عشر

المجموعات والمصفوفات

تسمح لغة **UNL** بتخزين أكثر من قيمة داخل متغير واحد باستخدام المجموعات (Sets) أو المصفوفات (Arrays)، مما يسهل التعامل مع كميات كبيرة من البيانات.

المجموعات (Set)

تُكتب باستخدام الأقواس الزاوية.

```
users = <"Ali", "Sara", "Omar">:  
  
Print(users)  
  
Unteck run.u(bg)
```

الوصول للعناصر

```
Print(users[0])  
  
Print(users[1])  
  
Unteck run.u(bg)
```

التكرار عليها

```
users = <"Ali","Sara","Omar">:  
  
for user in users={  
    Print(user)  
}:  
  
Unteck run.u(bg)
```

المصفوفات

```
numbers = [  
  
1,  
  
2,  
  
3,  
  
4,  
  
5  
  
]:  
  
Print(numbers)  
  
Unteck run.u(bg)
```

إرجاع مجموعة من دالة

```
func ids()={  
    return <1,2,3,4,5>  
}:  
Print(ids())  
Unteck run.u(bg)
```

إرجاع مصفوفة

```
func names()={  
    return [  
        "Ali",  
        "Sara",  
        "Omar"  
    ]  
}:  
Print(names())  
Unteck run.u(bg)
```

مثال عملي

```
scores = <95,80,40,100>:

for score in scores={

    If(score ≥ 50)={
        Print("ناجح")
    }:
    Else={
        Print("راسب")
    }:

}:

Unteck run.u(bg)
```

تُستخدم المجموعات والمصفوفات في تخزين المستخدمين، والمنتجات، ونتائج قواعد البيانات، والبيانات العلمية، وغيرها من التطبيقات.

الفصل الخامس عشر

معالجة الأخطاء (Error Handling)

قد تحدث أخطاء أثناء تنفيذ البرنامج، مثل القسمة على صفر، أو محاولة قراءة ملف غير موجود، أو الاتصال بخادم غير متاح.

لهذا توفر لغة UNL نظامًا لمعالجة الأخطاء باستخدام **Try** و **Catch**.

استخدام Try و Catch

```
Try={  
    result = 10 / 0  
}:  
Catch(error)={  
    Print(error)  
}:  
Unteck run.u(bg)
```

قراءة ملف

```
Try={  
    file.read("users.txt")  
}:  
Catch(error)={  
    Print("تعذر قراءة الملف")  
}:  
Unteck run.u(bg)
```

الاتصال بالشبكة

```
Try={  
    data = get("https://api.site.com")  
}:  
Catch(error)={  
    Print("فشل الاتصال")  
}:  
Unteck run.u(bg)
```

مثال عملي

```
Try={  
    user = database.select(  
        table="users"  
    )  
    Print(user)  
}:  
Catch(error)={  
    Print("حدث خطأ أثناء قراءة قاعدة البيانات")  
}:  
Unteck run.u(bg)
```

تساعد معالجة الأخطاء على منع توقف البرنامج بشكل مفاجئ، وتمنح المطور القدرة على عرض رسائل واضحة للمستخدم ومعالجة المشكلات بطريقة آمنة.

الفصل السادس عشر

البرمجة غير المتزامنة (Async Programming)

في بعض العمليات، مثل تنزيل الملفات أو إرسال طلبات الشبكة أو التواصل مع خدمات الذكاء الاصطناعي، قد يستغرق التنفيذ بعض الوقت.

بدلاً من إيقاف البرنامج بالكامل، توفر **UNL** نظام البرمجة غير المتزامنة باستخدام **async** و **await**.

إنشاء دالة Async

```
async func download()={  
    return api.get(url)  
}:  
  
Unteck run.u(bg)
```

انتظار النتيجة

```
data = await download()  
  
Print(data)  
  
Unteck run.u(bg)
```

مثال مع الذكاء الاصطناعي

```
un.ai

async func askAI()={
    return ai.ask("مرحباً")
}:

result = await askAI()

Print(result)

Unteck run.u(bg)
```

مثال مع تنزيل ملف

```
async func getFile()={
    return api.download(url)
}:

file = await getFile()

Print(file)

Unteck run.u(bg)
```

تشغيل عدة مهام

```
task1 = download()

task2 = askAI()

result1 = await task1

result2 = await task2

Print(result1)

Print(result2)

Unteck run.u(bg)
```

متى نستخدم Async؟

- عند تنزيل الملفات.
- عند رفع الملفات.
- عند استدعاء APIs.
- عند استخدام WebSocket.
- عند التواصل مع نماذج الذكاء الاصطناعي.
- عند تنفيذ العمليات التي قد تستغرق وقتاً طويلاً.

يساعد نظام **Async** في جعل التطبيقات أكثر سرعة واستجابة، خاصة في تطبيقات الويب، وتطبيقات الهاتف، والخوادم، والبرامج التي تعتمد على الشبكات.

الفصل السابع عشر

الوسيط un.ai

يعد **un.ai** الوسيط المسؤول عن جميع تقنيات الذكاء الاصطناعي داخل لغة **UNL**. ضمم ليُجعل التعامل مع نماذج الذكاء الاصطناعي بسيطًا، سواءً كان الهدف إنشاء مساعد ذكي، أو تحليل النصوص، أو التعلم من البيانات، أو بناء تطبيقات تعتمد على الذكاء الاصطناعي.

يمكن لهذا الوسيط التواصل مع نماذج مختلفة دون الحاجة إلى تغيير طريقة كتابة البرنامج، مما يمنح المطور مرونة كبيرة في اختيار النموذج المناسب.

استيراد الوسيط

```
Un.ai
```

```
Unteck run.u(bg)
```

()ai.ask

تستخدم لإرسال سؤال إلى نموذج الذكاء الاصطناعي والحصول على الإجابة.

```
Un.ai
```

```
result = ai.ask("ما عاصمة الجزائر")
```

```
Print(result)
```

```
Unteck run.u(bg)
```

ai.model

لتحديد النموذج المستخدم.

```
Un.ai  
  
ai.model = "gpt"  
  
Un.teck run.u(bg)
```

يمكن تغييره إلى أي نموذج مدعوم.

مثال:

```
ai.model = "gemini"
```

```
ai.model = "claude"
```

ai.temperature

تحدد درجة الإبداع في الإجابات.

```
ai.temperature = 0.2
```

إجابات دقيقة وأكثر ثباتًا.

```
ai.temperature = 0.8
```

إجابات أكثر إبداعًا.

(ai.learn

تسمح للنموذج بالتعلم من مجموعة بيانات بسيطة.

```

dataset =[

something={

"a=b",

"cat=animal",

"red=color"

}:

]:

result = ai.learn(dataset,"cat")

Print(result)

Unteck run.u(bg)

```

مثال: مترجم بسيط

```

Un.ai

result = ai.ask("Hello العربية ترجم كلمة")

Print(result)

Unteck run.u(bg)

```

مثال: شرح كود

```

Un.ai

code = "Print('Hello')"

result = ai.ask("اشرح هذا الكود" + code)

Print(result)

Unteck run.u(bg)

```

مثال: كتابة مقال

```
Un.ai

topic = "الطاقة الشمسية"

result = ai.ask("اكتب مقالاً عن" + topic)

Print(result)

Unteck run.U(bg)
```

مثال: إنشاء أسئلة

```
Un.ai

result = ai.ask("أنشئ خمسة أسئلة عن الفيزياء")

Print(result)

Unteck run.U(bg)
```

إنشاء مساعد ذكاء اصطناعي بسيط

```
Un.ai  
  
ai.model = "gpt"  
  
ai.temperature = 0.6  
  
Print("UNL AI Assistant")  
  
question = console.text=  
  
answer = ai.ask(question)  
  
Print(answer)  
  
Un.teck run.u(bg)
```

يقوم هذا المشروع بقراءة سؤال من المستخدم ثم إرساله إلى نموذج الذكاء الاصطناعي وعرض الإجابة مباشرة.

استخدامات un.ai

- إنشاء روبوتات محادثة.
- كتابة المقالات.
- ترجمة النصوص.
- تلخيص المحتوى.
- إنشاء أكواد برمجية.
- تحليل البيانات النصية.
- المساعدة التعليمية.
- بناء مساعدين أذكاء داخل التطبيقات.

يعد **un.ai** أحد أهم وسطاء لغة **UNL** لأنه يفتح المجال لبناء تطبيقات حديثة تعتمد على تقنيات الذكاء الاصطناعي بسهولة ووضوح.

الفصل الثامن عشر

الوسيط un.net

يُعد **un.net** الوسيط المسؤول عن جميع عمليات الشبكات والاتصال بالإنترنت داخل لغة **UNL**. ومن خلاله يمكن إرسال واستقبال البيانات، والتعامل مع واجهات البرمجة (APIs)، وتنزيل الملفات، ورفعها، وإنشاء اتصالات مباشرة باستخدام **WebSocket**. يهدف هذا الوسيط إلى توفير واجهة بسيطة وسهلة للتعامل مع الشبكات، مع الحفاظ على قوة ومرونة عالية.

استيراد الوسيط

```
Un.net  
Unteck run.u(bg)
```

جلب البيانات (GET)

تستخدم دالة **get()** لجلب البيانات من أي API.

```
Un.net  
  
data = get("https://api.site.com/users")  
  
Print(data)  
  
Unteck run.u(bg)
```

إرسال البيانات (POST)

تستخدم لإرسال بيانات إلى الخادم.

```
Un.net

user = {

name = "Ahmed"

age = 20

}:

post("https://api.site.com/users",user)

Unteck run.u(bg)
```

إضافة Headers

يمكن إرسال رؤوس الطلب بسهولة.

```
Un.net

api = {

token = "123"

type = "json"

}:

data = get("https://api.site.com",api)

Print(data)

Unteck run.u(bg)
```

api.get()

```
Un.net

users = api.get("/users")

Print(users)

Unteck run.u(bg)
```

()api.post

```
Un.net

login = {

  username="Ali"

  password="1234"

}:

api.post("/login",login)

Unteck run.u(bg)
```

()api.put

```
Un.net

user = {

  age = 22

}:

api.put("/user",user)

Unteck run.u(bg)
```

()api.delete

```
Un.net

api.delete("/user")

Unteck run.u(bg)
```

تنزيل الملفات

```
Un.net  
  
api.download("https://site.com/file.zip")  
  
Unteck run.u(bg)
```

رفع الملفات

```
Un.net  
  
api.upload("photo.png")  
  
Unteck run.u(bg)
```

WebSocket

إنشاء اتصال مباشر مع الخادم.

```
Un.net  
  
api.websocket("wss://site.com")  
  
Unteck run.u(bg)
```

إرسال رسالة عبر WebSocket

```
Un.net  
  
socket.send("Hello Server")  
  
Unteck run.u(bg)
```

استقبال الرسائل

```
Un.net

socket.onMessage(message)={

    Print(message)

}:

Unteck run.u(bg)
```

مشروع عملي

إنشاء تطبيق يعرض المستخدمين من API

```
Un.net

users = get("https://api.site.com/users")

Print("قائمة المستخدمين")

Print(users)

Unteck run.u(bg)
```

```
Un.net

Try={

    result = get("https://google.com")

    Print("الاتصال يعمل")

}:

Catch(error)={

    Print("لا يوجد اتصال بالإنترنت")

}:

Unteck run.u(bg)
```

استخدامات un.net

- استهلاك REST APIs.
- رفع الملفات.
- تنزيل الملفات.
- إرسال واستقبال البيانات.
- إنشاء تطبيقات متصلة بالإنترنت.
- إنشاء تطبيقات دردشة مباشرة.
- بناء خدمات تعتمد على WebSocket.
- ربط التطبيقات بقواعد البيانات السحابية.

يوفر **un.net** جميع الأدوات الأساسية التي يحتاجها المطور لإنشاء تطبيقات متصلة بالشبكة، مع واجهة موحدة وسهلة الاستخدام تناسب المبتدئين والمحترفين.

الفصل التاسع عشر

الوسيط un.cyber

يُعد **un.cyber** الوسيط المسؤول عن أدوات الأمن السيرياني داخل لغة **UNL**، حيث يوفر مجموعة من الدوال الجاهزة التي تساعد المطور على بناء تطبيقات أكثر أمانًا دون الحاجة إلى مكتبات خارجية.

يشمل هذا الوسيط التحقق من كلمات المرور، وتوليد كلمات مرور قوية، وإنشاء رموز OTP، والتحقق من البريد الإلكتروني، وتوليد قيم Hash، والتحقق من سلامة الملفات.

استيراد الوسيط

```
Un.cyber
```

```
Unteck run.u(bg)
```

توليد رمز OTP

تستخدم الدالة (**security.otp**) لإنشاء رمز تحقق يستخدم مرة واحدة.

```
Un.cyber
```

```
code = security.otp(6)
```

```
Print(code)
```

```
Unteck run.u(bg)
```

يمكن تغيير عدد الأرقام بسهولة.

```
code = security.otp(8)
```

التحقق من قوة كلمة المرور

```
Un.cyber

result = security.checkpass("MyPassword123")

Print(result)

Unteck run.u(bg)
```

إذا كانت كلمة المرور ضعيفة سيقتراح البرنامج تحسينها.

التحقق من البريد الإلكتروني

```
Un.cyber

email = "user@gmail.com"

result = security.email(email)

Print(result)

Unteck run.u(bg)
```

إنشاء Hash

يمكن تحويل أي نص إلى قيمة Hash.

```
Un.cyber

pass = "123456"

hash = crypto.hash(pass)

Print(hash)

Unteck run.u(bg)
```

اختيار نوع Hash

يدعم الوسيط عدة خوارزميات.

SHA-256

```
hash = cyber.hash.sha256("hello")
```

SHA-512

```
hash = cyber.hash.sha512("hello")
```

توليد كلمة مرور قوية

```
Un.cyber  
password = security.password(16)  
Print(password)  
Unteck run.U(bg)
```

يمكن تحديد أي طول.

```
security.password(32)
```

حماية تسجيل الدخول

بدلاً من حفظ كلمة المرور نفسها، يتم حفظ قيمة الـ Hash.

عند التسجيل

```
saved = cyber.hash("mypassword")
```

```
input = cyber.hash(userpass)

If(input == saved)={

    Print("تم تسجيل الدخول")

}:
```

التحقق من الملفات

يمكن التأكد من أن الملف لم يتم تعديله.

```
hash1 = cyber.filehash("app.apk")
```

بعد النقل أو التحميل

```
hash2 = cyber.filehash("app.apk")
```

التحقق

```
If(hash1 == hash2)={

    Print("الملف سليم")

}:
Else={

    Print("تم تعديل الملف")

}:

Unteck run.u(bg)
```

مشروع عملي

نظام تسجيل دخول آمن

```
Un.cyber

username = console.text=

password = console.pass=

saved = cyber.hash("123456")

input = cyber.hash(password)

If(input == saved)={

    Print("مرحباً " + username)

}:

Else={

    Print("كلمة المرور غير صحيحة")

}:

Unteck run.u(bg)
```

مشروع عملي

مولد كلمات مرور

```
Un.cyber

Print("Password Generator")

password = security.password(20)

Print(password)

Unteck run.u(bg)
```


التحقق من البريد الإلكتروني

```
Un.cyber

email = console.text=

If(security.email(email))={

    Print("البريد الإلكتروني صحيح")

}:
Else={

    Print("البريد الإلكتروني غير صحيح")

}:

Unteck run.u(bg)
```

استخدامات un.cyber

- إنشاء رموز OTP.
- حماية كلمات المرور.
- إنشاء Hash.
- التحقق من البريد الإلكتروني.
- التحقق من سلامة الملفات.
- إنشاء كلمات مرور قوية.
- بناء أنظمة تسجيل دخول آمنة.
- تطوير تطبيقات تعتمد على معايير أمنية حديثة.

يوفر **un.cyber** طبقة أمنية قوية داخل لغة **UNL**، مما يساعد المطور على بناء تطبيقات أكثر أمانًا دون الحاجة إلى كتابة خوارزميات التشفير يدويًا.

الفصل العشرون

الوسيط un.iot

يُعد **un.iot** الوسيط المسؤول عن برمجة أجهزة إنترنت الأشياء (IoT) داخل لغة **UNL**. يتيح للمطور الاتصال بالأجهزة الذكية، وقراءة بيانات الحساسات، والتحكم بالمعدات الإلكترونية بسهولة، مثل لوحات **ESP32** و **ESP8266** وغيرها من الأجهزة المدعومة. ضمّم هذا الوسيط ليكون بسيطًا للمبتدئين، وقويًا بما يكفي للمشاريع الاحترافية.

استيراد الوسيط

```
Un.iot  
Unteck run.u(bg)
```

الاتصال بجهاز

قبل التحكم بأي جهاز يجب إنشاء اتصال معه.

```
Un.iot  
  
device = iot.connect("ESP32")  
  
Unteck run.u(bg)
```

إنشاء جهاز

```
Un.iot  
  
lamp = iot.device("lamp")  
  
Unteck run.u(bg)
```

تشغيل جهاز

```
lamp.on()  
  
Unteck run.u(bg)
```

إيقاف جهاز

```
lamp.off()  
  
Unteck run.u(bg)
```

قراءة درجة الحرارة

```
Un.iot  
  
temp = sensor.temperature()  
  
Print(temp)  
  
Unteck run.u(bg)
```

قراءة الرطوبة

```
Un.iot

humidity = sensor.humidity()

Print(humidity)

Unteck run.u(bg)
```

الاستماع للحساسات

يمكن للبرنامج متابعة تغير قيمة الحساس مباشرة.

```
Un.iot

sensor.listen(temp)

Print(temp)

Unteck run.u(bg)
```

الأحداث (Events)

تنفذ أوامر عند تحقق شرط معين.

```
Un.iot

on.temperature(temp > 40)=[

    Print("درجة الحرارة مرتفعة")

]:

Unteck run.u(bg)
```

إرسال أوامر إلى الجهاز

```
Un.iot  
  
device = iot.connect("ESP32")  
  
iot.send(device,"ON")  
  
Unteck run.u(bg)
```

إرسال أكثر من أمر

```
Un.iot  
  
iot.send(device,"LED ON")  
  
iot.send(device,"FAN OFF")  
  
iot.send(device,"MOTOR START")  
  
Unteck run.u(bg)
```

مراقبة الحرارة باستمرار

```
Un.iot  
  
While.t(loop)+[  
    temp = sensor.temperature()  
    Print(temp)  
    Break(loop,20)  
]:  
  
Unteck run.u(bg)
```

مشروع عملي

نظام تحكم بمصباح ذكي

```
Un.iot

lamp = iot.device("Lamp")

lamp.on()

Print("تم تشغيل المصباح")

lamp.off()

Print("تم إيقاف المصباح")

Unteck run.u(bg)
```

مشروع عملي

مراقبة درجة الحرارة

```
Un.iot

While.t(tempLoop)+[

    temp = sensor.temperature()

    If(temp > 35)={

        Print("▲ حرارة مرتفعة")

    }:

    Break(tempLoop,30)

]:

Unteck run.u(bg)
```

التحكم بلوحة ESP32

```
Un.iot  
  
device = iot.connect("ESP32")  
  
iot.send(device,"LED ON")  
  
iot.send(device,"WAIT 3")  
  
iot.send(device,"LED OFF")  
  
Un.teck run.u(bg)
```

استخدامات un.iot

- التحكم بالمصابيح الذكية.
- التحكم بالمراوح.
- تشغيل وإيقاف المحركات.
- قراءة حساسات الحرارة.
- قراءة حساسات الرطوبة.
- قراءة حساسات الحركة.
- التحكم بالأجهزة المنزلية الذكية.
- مشاريع ESP32 و ESP8266.
- أنظمة الزراعة الذكية.
- أنظمة المنازل الذكية.
- الروبوتات.

ملخص الفصل

يوفر **un.iot** بيئة متكاملة لبناء تطبيقات إنترنت الأشياء داخل لغة **UNL**، مع واجهة برمجية بسيطة وواضحة تساعد المطور على إنشاء مشاريع ذكية دون الحاجة إلى التعامل مع تفاصيل معقدة منخفضة المستوى.

الفصل الحادي والعشرون

نظام Style

يعد **Style** نظام تنسيق الواجهات في لغة **UNL**، ويستخدم للتحكم في مظهر العناصر مثل الألوان، والخطوط، والأحجام، والمحاذاة، والحدود، والظلال، وغيرها من خصائص التصميم.

يشبه نظام **Style** ملفات **CSS** من حيث الفكرة، لكنه مُصمم ليتكامل مع لغة **UNL** بأسلوب أكثر بساطة وتنظيفًا.

إنشاء Style

تبدأ جميع أنماط التصميم بالكلمة:

```
Style [~  
~]:
```

داخلها يتم تعريف جميع خصائص التصميم.

القسم الرئيسي Major

يستخدم لتطبيق خصائص عامة على المشروع بالكامل.

```
Style [~  
:major={  
}  
~]:
```

لون الخلفية

```
Style [~  
  
:major={  
  
BG = #FFFFFF:  
  
}  
  
~]:
```

لون النص

```
Style [~  
  
:major={  
  
text = black:  
  
}  
  
~]:
```

اللون الرئيسي

يستخدم كلون افتراضي للتطبيق.

```
Style [~  
  
:major={  
  
main = #2196F3:  
  
}  
  
~]:
```

اتجاه النص

من اليمين إلى اليسار

```
text RTL
```

من اليسار إلى اليمين

```
text LTR
```

تلقائياً

```
text AUTO
```

يقوم النظام باختيار الاتجاه حسب أول حرف في النص.

الخط

تحديد نوع الخط.

```
text.f = Cairo:
```

الحدود

```
border = 2:
```

الظل

```
Shadow = true:
```

إنشاء Style لكلاس

```
Style [~  
  
@login={  
  
}  
  
~]:
```

لون النص

```
@login={  
  
text = blue:  
  
}
```

العرض

```
@login={  
  
width = 350:  
  
}
```

الارتفاع

```
@login={  
  height = 60:  
}
```

الحدود

```
@login={  
  border = 1:  
}
```

نوع الخط

```
@login={  
  f.f = Cairo:  
}
```

حجم الخط

```
@login={  
  f.s = 24:  
}
```

تصميم جميع الأزرار

```
.button={  
  
  BG = #2196F3:  
  
  text = white:  
  
}
```

تصميم جميع الحقول

```
.input={  
  
  border = 1:  
  
}
```

تصميم جميع النصوص

```
.text={  
  
  text = black:  
  
}
```

خاصية Display

لعرض أو إخفاء العناصر.

Flex

```
display = flex:
```

Block

```
display = block:
```

Inline

```
display = inline:
```

None

```
display = none:
```

أنواع الخطوط العربية

الاسم	الخط
cairo	Cairo
tajawal	Tajawal
almarai	Almarai
changa	Changa
kufam	Kufam
marhey	Marhey
amiri	Amiri

أنواع الخطوط الإنجليزية

الاسم	الخط
robot	Roboto

الاسم	الخط
open sans	Open Sans
mantserrat	Montserrat
poppins	Poppins
inter	Inter
playefair desplay	Playfair Display
fira code	Fira Code


```
Style [~  
  
:major={  
  
BG = #F7F7F7:  
  
text = black:  
  
main = #4CAF50:  
  
text RTL  
  
text.f = Cairo:  
  
}  
  
@login={  
  
width = 400:  
  
height = 60:  
  
text = blue:  
  
border = 2:  
  
f.s = 20:  
  
}  
  
.button={  
  
BG = #4CAF50:  
  
text = white:  
  
}  
  
~]:
```

تصميم صفحة تسجيل دخول

```
Style [~
:major={
BG = #EEEEEE:
text = black:
text RTL
text.f = Cairo:
}
@login={
width = 420:
height = 60:
border = 2:
}
.button={
BG = green:
text = white:
}
~]:
```

بعد تطبيق هذا التصميم على عناصر الواجهة سيحصل التطبيق على مظهر موحد واحترافي.

استخدامات Style

- تصميم صفحات الويب.
- تصميم تطبيقات سطح المكتب.
- تصميم تطبيقات الهاتف.

- تخصيص الخطوط.
- إنشاء واجهات احترافية.
- التحكم الكامل في ألوان المشروع.
- إنشاء قوالب جاهزة لإعادة الاستخدام.

ملخص الفصل

يوفر نظام **Style** في لغة **UNL** طريقة سهلة ومرنة لتنسيق واجهات المستخدم، حيث يمكن التحكم في جميع عناصر التصميم من مكان واحد، مما يجعل بناء الواجهات أسرع وأكثر تنظيماً.

الفصل الثاني والعشرون

قواعد البيانات (Database)

تُعد قواعد البيانات من أهم مكونات التطبيقات الحديثة، إذ تُستخدم لتخزين المعلومات واسترجاعها وإدارتها بطريقة منظمة. توفر لغة **UNL** نظامًا بسيطًا ومرنًا للتعامل مع قواعد البيانات، مما يسمح ببناء تطبيقات حقيقية مثل أنظمة المستخدمين، والمتاجر الإلكترونية، ولوحات التحكم.

فتح قاعدة بيانات

قبل تنفيذ أي عملية يجب فتح قاعدة البيانات.

```
db.open("users.db")

Unteck run.u(bg)
```

إدراج البيانات

تستخدم الدالة **db.insert()** لإضافة سجل جديد.

```
db.insert({

  name = "Ali"

  age = 20

}):

Unteck run.u(bg)
```

قراءة البيانات

يمكن استرجاع جميع البيانات بسهولة.

```
users = db.select("users")

Print(users)

Unteck run.u(bg)
```

قراءة سجل واحد

```
user = db.select("users",1)

Print(user)

Unteck run.u(bg)
```

تحديث البيانات

```
db.update(

    table="users",

    id=1,

    data={

        age=25

    }

)

Unteck run.u(bg)
```

حذف البيانات

```
db.delete(  
    table="users",  
    id=1  
)  
  
Unteck run.u(bg)
```

البحث داخل قاعدة البيانات

```
result = db.select(  
    table="users",  
    where={  
        name="Ali"  
    }  
)  
  
Print(result)  
  
Unteck run.u(bg)
```

مثال عملي

إنشاء سجل طالب

```
db.open("school.db")

db.insert({

    name="Ahmed"

    age=15

    level="First"

}):

Unteck run.u(bg)
```

مثال عملي

عرض جميع الطلاب

```
students = db.select("school")

Print(students)

Unteck run.u(bg)
```

```
db.open("school.db")

db.insert({

    name="Ali"

    age=16

}):

db.insert({

    name="Sara"

    age=15

}):

students = db.select("school")

Print(students)

Unteck run.u(bg)
```

استخدامات قواعد البيانات

- أنظمة تسجيل الدخول.
- المتاجر الإلكترونية.
- تطبيقات المدارس.
- تطبيقات المستشفيات.
- برامج إدارة الشركات.
- تطبيقات الهاتف.
- تطبيقات سطح المكتب.
- مواقع الويب.

ملخص الفصل

يقدم نظام قواعد البيانات في **UNL** طريقة سهلة للتعامل مع البيانات دون تعقيد، مما يسمح ببناء تطبيقات احتراافية تعتمد على تخزين المعلومات وإدارتها بكفاءة.

الفصل الثالث والعشرون

تقسيم المشاريع باستخدام Link.file

كلما كبر المشروع، أصبح من الصعب وضع جميع الأكواد داخل ملف واحد. لذلك توفر لغة UNL نظام **Link.file** لتقسيم المشروع إلى عدة ملفات مترابطة، مما يجعل الكود أكثر تنظيماً وأسهل في الصيانة.

ربط ملف

```
Link.file(math.u)=on  
Unteck run.u(bg)
```

إلغاء ربط ملف

```
Link.file(math.u)=off  
Unteck run.u(bg)
```

مثال

ملف **math.u**

```
func area(w,h)={  
    return w*h  
}:  
Unteck run.u(bg)
```

```
Link.file(math.u)=on  
  
result = area(20,10)  
  
Print(result)  
  
Unteck run.u(bg)
```

تقسيم مشروع كبير

```
Project/  
|  
├─ main.u  
├─ math.u  
├─ users.u  
├─ login.u  
├─ ai.u  
├─ style.u  
└─ database.u
```

كل ملف مسؤول عن جزء معين من المشروع.

مثال عملي

```
Link.file(users.u)=on  
  
Link.file(database.u)=on  
  
Link.file(login.u)=on  
  
start()  
  
Unteck run.u(bg)
```

فوائد Link.file

- تنظيم المشروع.
 - تسهيل الصيانة.
 - إعادة استخدام الكود.
 - تقسيم العمل بين عدة مطورين.
 - تسريع تطوير المشاريع الكبيرة.
-

مشروع عملي

إنشاء مشروع آلة حاسبة

```
Calculator/  
|  
├─ main.u  
├─ math.u  
├─ style.u  
└─ history.u
```

يقوم **main.u** بتشغيل المشروع، بينما يحتوي **math.u** على العمليات الحسابية، و**style.u** على تصميم الواجهة، و**history.u** على سجل العمليات الحسابية.

ملخص الفصل

يساعد **Link.file** على تحويل المشروع من ملف واحد كبير إلى مجموعة ملفات منظمة، وهو أسلوب يُستخدم في جميع المشاريع الاحترافية مهما كان حجمها.

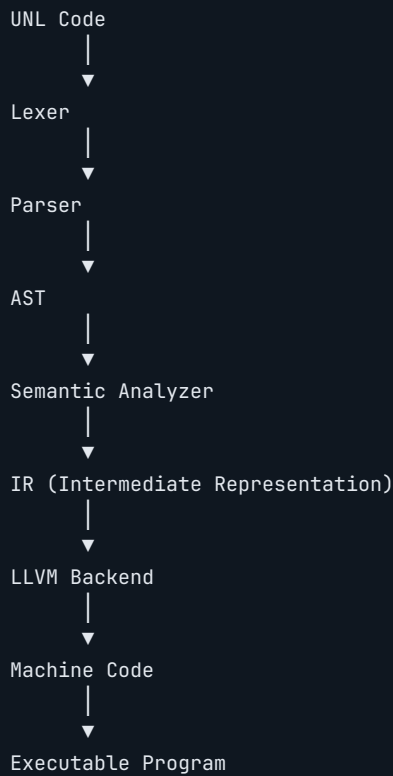
الفصل الرابع والعشرون

كيف يعمل Compiler في لغة UNL

عند كتابة برنامج بلغة **UNL** فإن الحاسوب لا يستطيع تنفيذه مباشرة، لأن المعالج لا يفهم إلا لغة الآلة (Machine Code). لذلك يأتي دور **Compiler** الذي يحول كود UNL إلى تعليمات قابلة للتنفيذ.

لا تتم هذه العملية في خطوة واحدة، بل تمر بعدة مراحل، حيث تؤدي كل مرحلة وظيفة محددة حتى يصبح البرنامج جاهزاً للتشغيل.

مخطط عمل الـ Compiler



لماذا توجد كل هذه المراحل؟

تقسيم عملية الترجمة إلى مراحل يجعل بناء اللغة أكثر استقراراً، ويسهل اكتشاف الأخطاء، كما يسمح بإضافة مزايا جديدة مستقبلاً دون إعادة تصميم المترجم بالكامل.

مثال مبسط

لنفترض وجود البرنامج التالي:

```
Print("Welcome in UNL")  
  
Unteck run.u(bg)
```

لن ينفذ مباشرة، بل ينتقل بين جميع المراحل السابقة حتى يتحول في النهاية إلى برنامج يعمل على نظام التشغيل.

فوائد Compiler

- تحسين سرعة التنفيذ.
- اكتشاف الأخطاء قبل تشغيل البرنامج.
- إنتاج ملفات تنفيذية مستقلة.
- تحسين استهلاك الذاكرة.
- تحسين أداء التطبيقات الكبيرة.
- دعم أنظمة تشغيل متعددة.
- إمكانية تطوير اللغة مستقبلاً بسهولة.

أنواع المخرجات

يمكن أن ينتج Compiler عدة أنواع من الملفات حسب طريقة البناء، مثل:

- ملفات تنفيذية.
- ملفات خاصة بالمكتبات.
- ملفات وسيطة.
- ملفات مخصصة للتصحيح.

العلاقة بين Interpreter و Compiler

يعتمد **Interpreter** على تنفيذ التعليمات أثناء القراءة، بينما يقوم **Compiler** بترجمة البرنامج أولاً ثم تشغيل النسخة الناتجة.

تتجه لغة **UNL** إلى استخدام Compiler للحصول على أداء أعلى وإنتاج تطبيقات احترافية.

ملخص الفصل

يُعد Compiler القلب الحقيقي للغة البرمجة، فهو المسؤول عن تحويل الكود المكتوب بلغة UNL إلى برنامج يستطيع الحاسوب فهمه وتنفيذه بكفاءة.

الفصل الخامس والعشرون

Lexer

يُعد **Lexer** أول مرحلة من مراحل بناء مترجم لغة **UNL**، وهو المسؤول عن قراءة الكود الذي يكتبه المبرمج وتحويله إلى مجموعة من الرموز المنظمة، والتي تُعرف باسم **Tokens**.

لا يهتم الـ **Lexer** بفهم معنى البرنامج، بل يهتم فقط بتقسيم النص إلى أجزاء يمكن للمراحل التالية التعامل معها.

مخطط عمل Lexer



مثال

لنفترض وجود البرنامج التالي:

```
Print("Welcome")  
  
Unteck run.u(bg)
```

سيقوم الـ **Lexer** بتحويله إلى شيء يشبه:


```
PRINT  
(  
STRING  
)  
END_PROGRAM
```

كل عنصر من العناصر السابقة يسمى **Token**.

أنواع الـ Tokens

يتعرف الـ Lexer على أنواع مختلفة من الرموز، مثل:

- الكلمات المحجوزة (Keywords)
- أسماء المتغيرات (Identifiers)
- الأرقام (Numbers)
- النصوص (Strings)
- العمليات الحسابية
- العمليات المنطقية
- الأقواس
- الفواصل
- التعليقات
- نهاية البرنامج

مثال آخر

```
age = 15  
  
Unteck run.u(bg)
```

يتم تقسيمه إلى:

```
Identifier
Assignment
Number
End Program
```

معالجة الفراغات

الفراغات والأسطر الجديدة تساعد على تنظيم الكود، لكنها غالبًا لا تؤثر على المعنى، لذلك يتجاهلها الـ Lexer في معظم الحالات.

مثال:

```
Print("Hello")
```

9

```
Print ( "Hello" )
```

كلاهما ينتجان الرموز نفسها.

التعليقات

التعليقات ليست جزءًا من تنفيذ البرنامج، لذلك يتجاوزها الـ Lexer أثناء التحليل.

مثال:

```
// هذا تعليق
Print("Hello")
Unteck run.u(bg)
```

لن ينتج Token خاصًا بالتعليق.

النصوص

يدعم UNL أكثر من طريقة لكتابة النصوص، ويعتبرها الـ Lexer جميعًا نصوصًا.

مثال:

```
"Hello"  
'Hello'  
`Hello`
```

وجميعها تتحول إلى Token من نوع:

```
STRING
```

لماذا يعتبر Lexer مهمًا؟

لأنه يمثل البوابة الأولى للمترجم، فإذا فشل في قراءة الكود بشكل صحيح فلن تتمكن أي مرحلة بعده من العمل.

كما يساعد في:

- اكتشاف الرموز غير الصحيحة.
- تقسيم الكود بسرعة.
- تسهيل عمل Parser.
- تحسين سرعة المترجم.

رسم توضيحي

```
Print("UNL")
```



```
Lexer
```



```
PRINT  
(  
STRING  
)
```

ملخص الفصل

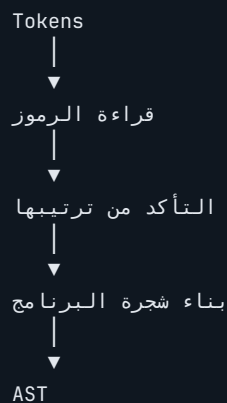
يقوم **Lexer** بتحويل الكود المكتوب بلغة UNL إلى مجموعة من الرموز المنظمة (Tokens)، والتي تُستخدم لاحقًا بواسطة **Parser** لفهم بنية البرنامج. ويُعد هذه المرحلة الأساس الذي تعتمد عليه جميع المراحل التالية من عملية الترجمة.

الفصل السادس والعشرون

Parser

بعد أن ينتهي **Lexer** من تقسيم الكود إلى **Tokens**، تبدأ المرحلة الثانية وهي **Parser**. وظيفة الـ **Parser** هي قراءة الرموز التي أنتجها الـ **Lexer**، ثم التأكد من أنها مكتوبة وفق قواعد لغة **UNL**، وبعد ذلك يبني هيكلًا منظمًا يمثل البرنامج. يمكن اعتبار الـ **Parser** مهندسًا يراجع المخطط قبل البدء في البناء.

مخطط عمل Parser



مثال بسيط

لنفترض وجود الكود التالي:

```
Print("Hello")  
  
Unteck run.u(bg)
```

بعد مرحلة **Lexer** تصبح الرموز:

```
PRINT  
(  
  STRING  
)  
END_PROGRAM
```

ثم يقوم Parser بتحويلها إلى بنية مفهومة تمثل عملية استدعاء الدالة Print مع النص "Hello".

مثال آخر

```
age = 18  
  
Print(age)  
  
Unteck run.U(bg)
```

يفهم Parser أن:

- يوجد متغير اسمه **age**.
 - القيمة المسندة إليه هي **18**.
 - ثم يتم تمرير المتغير إلى الدالة **Print**.
- أي أنه لا يرى الكلمات فقط، بل يفهم العلاقة بينها.

ماذا يتحقق Parser؟

يتأكد Parser من عدة أمور، مثل:

- إغلاق جميع الأقواس.
- صحة ترتيب الجمل.
- اكتمال أوامر **If Else**.
- صحة تعريف الدوال.
- صحة الحلقات.
- صحة الكائنات.
- صحة المصفوفات والمجموعات.
- صحة نهاية البرنامج.

مثال على خطأ

```
Print("Hello"  
  
Unteck run.u(bg)
```

نسي المبرمج إغلاق القوس.

سيكتشف Parser الخطأ ويوقف عملية الترجمة.

مثال آخر

```
If(age > 18)  
  
Print("Adult")  
  
Unteck run.u(bg)
```

إذا كانت صيغة الشرط لا تطابق قواعد UNL، فإن Parser يبلغ بوجود خطأ نحوي.

لماذا لا يقوم Lexer بذلك؟

لأن Lexer لا يهتم إلا بتقسيم النص إلى رموز.

أما Parser فهو الذي يفهم كيفية ارتباط هذه الرموز معًا.

رسم توضيحي

```
Print("Hello")  
  |  
  ▼  
  Lexer  
  |  
  PRINT ( STRING )  
  |  
  ▼  
  Parser  
  |  
  ▼  
  Call Print  
  |  
  STRING
```

فوائد Parser

- اكتشاف الأخطاء النحوية.
- بناء هيكل منظم للبرنامج.
- تجهيز البيانات لبناء AST.
- ضمان توافق الكود مع قواعد لغة UNL.
- تسهيل عمل المراحل التالية.

العلاقة بين Parserg Lexer

```
graph TD; UNL_Code[UNL Code] --> Lexer; Lexer --> Tokens; Tokens --> Parser; Parser --> AST;
```

لا يمكن لـ Parser العمل دون Tokens، كما أن Lexer لا يستطيع فهم بنية البرنامج، لذلك يعملان معًا في تسلسل ثابت.

ملخص الفصل

يحلل **Parser** الرموز التي ينتجها **Lexer**، ويتأكد من أنها مرتبة وفق قواعد لغة **UNL**. وبعد نجاح هذه المرحلة، يصبح البرنامج جاهزًا للانتقال إلى **AST**، حيث تتحول البنية النحوية إلى شجرة تمثل البرنامج بصورة منظمة يسهل على المترجم التعامل معها.

الفصل السابع والعشرون

AST (Abstract Syntax Tree)

بعد أن ينتهي **Parser** من التأكد من صحة الكود، تبدأ المرحلة التالية وهي **AST**، وهي اختصار لعبارة **Abstract Syntax Tree**، أي شجرة البنية المجردة.

تمثل الـ AST البرنامج على شكل شجرة، حيث تمثل كل عقدة (Node) جزءًا من البرنامج، مثل دالة أو متغير أو عملية حسابية أو شرط.

هذه الشجرة لا تهتم بطريقة كتابة الكود، بل تهتم بالمعنى والبنية.

لماذا نستخدم AST؟

يكتب المبرمج الكود بطريقة تناسبه، لكن المترجم يحتاج إلى تمثيل منظم يمكن التعامل معه بسهولة.

لهذا السبب يحول Parser البرنامج إلى AST.

مخطط المراحل

```
graph TD; A[UNL Code] --> B[Lexer]; B --> C[Tokens]; C --> D[Parser]; D --> E[AST];
```

مثال بسيط

لنفترض وجود البرنامج التالي:

```
Print("Hello")  
  
Unteck run.u(bg)
```

تمثيله داخل AST يكون بصورة مبسطة:

```
Program  
├── Print  
│   └── "Hello"
```

لاحظ أن الشجرة تهتم بالعلاقة بين العناصر، وليس بشكل الأقواس أو الفراغات.

مثال آخر

```
result = 5 + 3  
  
Unteck run.u(bg)
```

تصبح الشجرة:

```
Assignment  
├── result  
└── +  
    ├── 5  
    └── 3
```

يتضح أن العملية الحسابية أصبحت عقدة مستقلة داخل الشجرة.

مثال مع شرط

```
If(age ≥ 18)={  
    Print("Adult")  
}:  
  
Unteck run.u(bg)
```

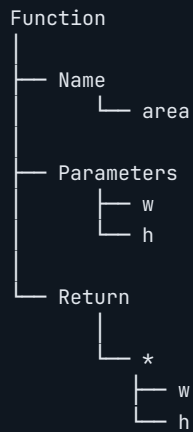
تمثيل مبسط:

```
If  
├── Condition  
│   └── age ≥ 18  
└── Body  
    └── Print  
        └── "Adult"
```

مثال مع دالة

```
func area(w,h)={  
    return w*h  
}:  
  
Unteck run.u(bg)
```

قد يصبح شكل الشجرة:



مميزات AST

- تمثل البرنامج بطريقة منظمة.
- تسهل تحليل الكود.
- تسهل تحسين الأداء.
- تساعد في اكتشاف الأخطاء.
- تستخدم في جميع المراحل التالية.

الفرق بين Parser وAST

Parser يتحقق من صحة ترتيب الجمل.

أما AST فهو يمثل البرنامج في صورة شجرة يمكن للمترجم التعامل معها.

Print("UNL")



Lexer



Tokens



Parser



AST



Program



Print()



"UNL"

ماذا يحدث بعد AST؟

بعد بناء الشجرة، تنتقل إلى مرحلة جديدة تسمى **Semantic Analyzer**.

في هذه المرحلة لا يتم فحص شكل الكود، بل يتم فحص معناه، مثل:

- هل المتغير موجود؟
- هل نوع البيانات صحيح؟
- هل الدالة معرفة؟
- هل عملية الإسناد صحيحة؟

ملخص الفصل

تمثل **AST** البرنامج على شكل شجرة منظمة تحتوي على جميع عناصره والعلاقات بينها. وتعد هذه الشجرة الأساس الذي تعتمد عليه بقية مراحل المترجم، مثل التحليل الدلالي وتحسين الكود وتوليد التعليمات النهائية.

الفصل الثامن والعشرون

Semantic Analyzer

بعد أن يبني **Parser** شجرة البرنامج (**AST**)، تبدأ مرحلة جديدة تُسمى **Semantic Analyzer** أو **المحلل الدلالي**. في هذه المرحلة لا يهتم المترجم بشكل الكود، لأن ذلك تم التحقق منه مسبقاً، وإنما يهتم بمعنى الكود، وهل التعليمات منطقية ويمكن تنفيذها بالفعل. بمعنى آخر، يسأل المحلل الدلالي:

هل هذا البرنامج صحيح من ناحية المعنى، وليس فقط من ناحية الكتابة؟

مكان Semantic Analyzer داخل المترجم

```
graph TD; UNL[UNL Code] --> Lexer[Lexer]; Lexer --> Parser[Parser]; Parser --> AST[AST]; AST --> SA[Semantic Analyzer];
```

ماذا يفعل؟

يقوم المحلل الدلالي بعدة مهام مهمة، منها:

- التأكد من أن المتغيرات معرفة.

- التأكد من أن الدوال موجودة.
- التحقق من أنواع البيانات.
- التأكد من صحة المعاملات.
- التحقق من قيمة return.
- التأكد من صحة استدعاء الدوال.
- التحقق من نطاق المتغيرات (Scope).

مثال رقم 1

```
Print(name)

Unteck run.u(bg)
```

في هذا المثال لم يتم تعريف **name**.
سيكتشف المحلل الدلالي ذلك ويبلغ بوجود خطأ.

مثال رقم 2

```
age = 18

Print(age)

Unteck run.u(bg)
```

هنا كل شيء صحيح، لأن المتغير موجود قبل استخدامه.

التحقق من الدوال

```
result = area(10,20)

Unteck run.u(bg)
```

إذا كانت الدالة **area** غير موجودة، فسوف يظهر خطأ.

أما إذا كانت معرفة مسبقاً، فيسمح البرنامج بالانتقال إلى المرحلة التالية.

التحقق من نوع البيانات

مثال:

```
age = "Ali"

number = age + 10

Unteck run.U(bg)
```

قد تكون هذه العملية غير صحيحة إذا كانت اللغة لا تسمح بجمع النصوص مع الأرقام.

لذلك يتحقق المحلل الدلالي من توافق الأنواع.

التحقق من Return

```
func area()={

    return 100

}:
```

يتأكد المحلل الدلالي من أن قيمة **return** مناسبة للدالة، وأنها موجودة في المكان الصحيح.

التحقق من Scope

```
func test()={  
    x = 10  
}:  
Print(x)  
Unteck run.u(bg)
```

المتغير **x** موجود داخل الدالة فقط.

لذلك سيكتشف المحلل الدلالي أن استخدامه خارج الدالة غير صحيح.

التحقق من المعاملات

```
func sum(a,b)={  
    return a+b  
}:  
sum(5)  
Unteck run.u(bg)
```

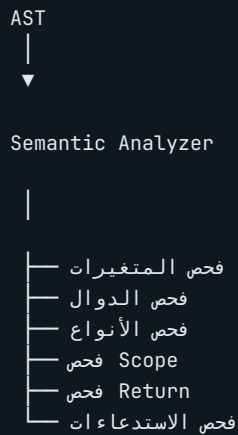
الدالة تحتاج إلى معاملين، لكن تم إرسال معامل واحد فقط.

سيتم الإبلاغ عن الخطأ.

لماذا تعتبر هذه المرحلة مهمة؟

بدون المحلل الدلالي قد ينجح البرنامج في المرور من Parser رغم احتوائه على أخطاء منطقية تمنع تشغيله بشكل صحيح.

لهذا السبب تعد هذه المرحلة من أهم مراحل بناء أي مترجم احترافي.



ماذا يحدث بعد ذلك؟

بعد التأكد من صحة البرنامج من الناحية الدلالية، يصبح جاهزًا للتحويل إلى تمثيل بسيط يسمى **IR (Intermediate Representation)**.

وهذا التمثيل لا يعتمد على لغة UNL أو على نظام التشغيل، بل يمثل البرنامج بطريقة داخلية يمكن تحسينها قبل إنتاج الكود النهائي.

ملخص الفصل

يقوم **Semantic Analyzer** بالتحقق من معنى البرنامج، وليس من طريقة كتابته. فهو يتأكد من صحة المتغيرات والدوال والأنواع ونطاق المتغيرات والاستدعاءات وقيم الإرجاع، مما يجعل البرنامج جاهزاً للانتقال إلى مرحلة **IR** بثقة ودقة.

الفصل التاسع والعشرون

IR (Intermediate Representation)

بعد أن يجتاز البرنامج مرحلة **Semantic Analyzer** بنجاح، ينتقل إلى مرحلة جديدة تُسمى **IR**، وهي اختصار لعبارة **Intermediate Representation**، أي التمثيل الوسيط.

يُعد IR لغة داخلية يفهمها المترجم، ولا يكتبها المبرمج، ولا تظهر للمستخدم أثناء تطوير البرامج.

الهدف منها هو تحويل البرنامج إلى صيغة موحدة يمكن تحسينها قبل إنتاج البرنامج النهائي.

مكان IR داخل المترجم



لماذا نستخدم IR؟

قد يكون البرنامج مكتوبًا بطريقة مختلفة، لكن يؤدي النتيجة نفسها.

وجود IR يسمح للمترجم بتحويل جميع هذه الطرق إلى تمثيل واحد، ثم يبدأ بتحسينه.

وبذلك يصبح إنتاج الكود النهائي أسرع وأكثر كفاءة.

مثال توضيحي

كتب المترجم:

```
result = 5 + 3  
  
Unteck run.u(bg)
```

لن ينتقل مباشرة إلى لغة الآلة، بل يتحول أولاً إلى تمثيل داخلي يشبه:

```
LOAD 5  
LOAD 3  
ADD  
STORE result
```

هذا مجرد مثال توضيحي، وليس التمثيل الحقيقي الذي تستخدمه UNL.

مثال آخر

```
Print("Hello")  
  
Unteck run.u(bg)
```

قد يتحول إلى خطوات داخلية مثل:

```
LOAD STRING  
CALL PRINT  
END
```

مرة أخرى، هذا المثال للتوضيح فقط.

تحسين الكود

من أهم فوائده IR أنه يسمح للمترجم بتحسين البرنامج.

مثال:

```
x = 5 + 5
```

```
Unteck run.u(bg)
```

قد يكتشف المترجم أن الناتج معروف مسبقًا، فيستبدل العملية بالقيمة النهائية مباشرة.

بدل تنفيذ عمليتي تحميل وجمع، يصبح التنفيذ أسرع.

إزالة التعليمات غير المستخدمة

مثال:

```
x = 10
```

```
x = 20
```

```
Print(x)
```

```
Unteck run.u(bg)
```

القيمة الأولى لم تعد مستخدمة.

يمكن للمترجم حذفها أثناء مرحلة IR لتقليل حجم البرنامج.

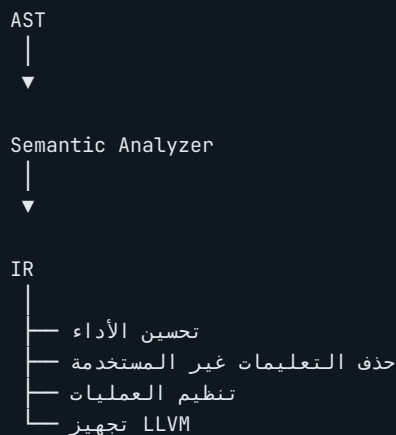
تحسين الشروط

إذا كان الشرط معروف النتيجة أثناء الترجمة، فقد يتم حذف الفرع غير المستخدم.

وهذا يقلل حجم الكود ويحسن سرعة التنفيذ.

تحسين الحلقات

قد يقوم المترجم بتحسين بعض الحلقات لتقليل عدد التعليمات المنفذة، مع الحفاظ على النتيجة نفسها.



لماذا لا ينتقل البرنامج مباشرة إلى LLVM؟

لأن LLVM يحتاج إلى تمثيل منظم وواضح.

لذلك يعمل IR كمرحلة وسيطة تجعل عملية التحويل إلى LLVM أسهل وأكثر كفاءة.

فوائد IR

- تحسين سرعة التنفيذ.
- تقليل حجم البرنامج.
- إزالة التعليمات غير الضرورية.
- تنظيم العمليات الحسابية.
- تجهيز البرنامج للمرحلة التالية.
- تحسين جودة الملفات التنفيذية.

ماذا يحدث بعد IR؟

بعد الانتهاء من جميع عمليات التحسين، يصبح البرنامج جاهزاً للانتقال إلى **LLVM Backend**.

وهناك يبدأ تحويل البرنامج إلى تعليمات منخفضة المستوى يمكن تحويلها لاحقاً إلى **Machine Code**.

ملخص الفصل

يمثل **IR** المرحلة التي يتحول فيها البرنامج إلى صيغة داخلية مستقلة عن لغة البرمجة ونظام التشغيل، مما يسمح بإجراء تحسينات متعددة قبل إنتاج الكود النهائي، ويُعد حلقة الوصل الأساسية بين التحليل الدلالي ومرحلة **LLVM**.

الفصل الثلاثون

LLVM Backend

بعد انتهاء مرحلة **IR (Intermediate Representation)**، يصبح البرنامج جاهزًا للانتقال إلى إحدى أهم مراحل المترجم، وهي **LLVM Backend**.

تُعد هذه المرحلة مسؤولة عن تحويل التمثيل الوسيط (**IR**) إلى تعليمات منخفضة المستوى يمكن للمعالج فهمها بعد تحويلها إلى **Machine Code**.

وباختصار، فإن LLVM هو الجسر الذي ينقل البرنامج من عالم البرمجة إلى عالم المعالج.

مكان LLVM داخل المترجم

```
graph TD; UNL[UNL Code] --> Lexer[Lexer]; Lexer --> Parser[Parser]; Parser --> AST[AST]; AST --> SA[Semantic Analyzer]; SA --> IR[IR]; IR --> LLVM[LLVM Backend]; LLVM --> MC[Machine Code];
```

ما هو LLVM؟

LLVM هو مشروع مفتوح المصدر يُستخدم في العديد من لغات البرمجة الحديثة. يوفر مجموعة من الأدوات التي تساعد المترجمات على إنتاج برامج سريعة وعالية الكفاءة دون الحاجة إلى كتابة مولد تعليمات خاص بكل معالج. ولهذا السبب تعتمد عليه لغات كثيرة في بناء مترجماتها.

لماذا اختارت UNL استخدام LLVM؟

يساعد LLVM لغة UNL على تحقيق عدة أهداف، منها:

- إنتاج برامج سريعة.
- دعم أنظمة تشغيل متعددة.
- دعم معالجات مختلفة.
- الاستفادة من تقنيات تحسين الأداء.
- تقليل حجم الملفات التنفيذية.
- تسهيل تطوير المترجم مستقبلاً.

مخطط العمل

```
graph TD; IR --> LLVM_Backend[LLVM Backend]; LLVM_Backend --> Machine_Code[Machine Code]; Machine_Code --> Executable
```

مثال توضيحي

لنفترض أن البرنامج هو:

```
Print("Welcome")

Unteck run.U(bg)
```

تمر عملية البناء كما يلي:

```
UNL Code
↓
Lexer
↓
Parser
↓
AST
↓
Semantic Analyzer
↓
IR
↓
LLVM
↓
Machine Code
↓
برنامج يعمل
```

تحسين الأداء

لا يقتصر دور LLVM على تحويل الكود فقط، بل يقوم أيضًا بتحسينه.

ومن أمثلة ذلك:

- تقليل عدد التعليمات.
- تحسين العمليات الحسابية.
- إزالة التعليمات غير المستخدمة.
- تحسين استخدام الذاكرة.
- تحسين سرعة التنفيذ.

دعم الأنظمة

بفضل LLVM يمكن بناء برامج تعمل على أنظمة متعددة مثل:

- Windows ▪
- Linux ▪
- macOS ▪

كما يمكن استهداف معالجات مختلفة دون الحاجة إلى إعادة كتابة المترجم لكل معالج.

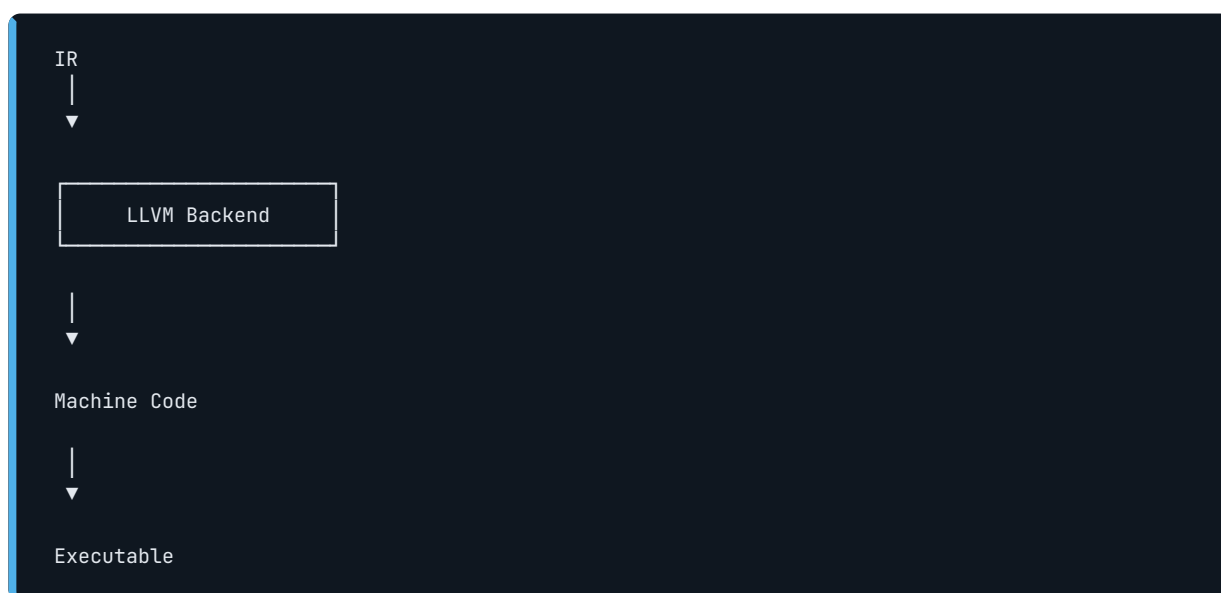
لماذا لا ينتج Machine Code مباشرة؟

لو قام المترجم بإنتاج تعليمات المعالج مباشرة، فسيحتاج إنشاء مولد مختلف لكل معمارية.

أما باستخدام LLVM، فيكفي إنتاج IR، بينما يتولى LLVM بقية العمل.

وهذا يجعل تطوير لغة UNL أسهل وأكثر قابلية للتوسع.

رسم توضيحي



المرحلة الأخيرة

بعد انتهاء LLVM من عمله، يتم إنشاء **Machine Code**، وهو الكود الذي يفهمه المعالج مباشرة.

ثم يتم بناء الملف التنفيذي الذي يستطيع المستخدم تشغيله على جهازه.

مخطط كامل لرحلة البرنامج

```
graph TD; UNL[UNL Code] --> Lexer[Lexer]; Lexer --> Parser[Parser]; Parser --> AST[AST]; AST --> SA[Semantic Analyzer]; SA --> IR[IR]; IR --> LLVM[LLVM Backend]; LLVM --> MC[Machine Code]; MC --> EP[Executable Program];
```

ملخص الفصل

يُعد **LLVM Backend** المرحلة التي تتحول فيها البنية الداخلية للبرنامج إلى تعليمات منخفضة المستوى تمهيدًا لإنتاج **Machine Code**. وبفضل LLVM تستطيع لغة **UNL** الاستفادة من تقنيات متقدمة لتحسين الأداء، ودعم أنظمة تشغيل متعددة، وإنتاج برامج تنفيذية عالية الكفاءة دون الحاجة إلى بناء مولد تعليمات خاص لكل معالج.

الفصل الحادي والثلاثون

UNL Studio

بعد تعلم أساسيات لغة **UNL** وفهم كيفية عمل المترجم، حان الوقت للتعرف على بيئة التطوير الرسمية للغة، وهي **UNL Studio**. تم تصميم **UNL Studio** ليكون المكان الذي يكتب فيه المطور البرامج، ويشغلها، ويصحح الأخطاء، ويبنى التطبيقات، ويُدير المشاريع من واجهة واحدة.

سواء كنت مبتدئاً أو محترفاً، ستجد جميع الأدوات التي تحتاجها داخل البرنامج.

ما هو UNL Studio؟

UNL Studio هو بيئة تطوير متكاملة (IDE) مخصصة للغة **UNL**، تجمع بين محرر الأكواد، والمترجم، ووحدة بناء التطبيقات، وأدوات الذكاء الاصطناعي، ومدير الحزم.

الهدف منه هو توفير تجربة تطوير سهلة وسريعة ومنظمة.

واجهة البرنامج

عند تشغيل البرنامج ستظهر الواجهة الرئيسية، وتتكون من عدة أجزاء.



شريط القوائم (Menu Bar)

يحتوي الشريط العلوي على أهم أوامر البرنامج.

- File ▾
- Edit ▾
- Run ▾
- Build ▾
- Tools ▾

ومن خلاله يمكن إنشاء المشاريع، وفتح الملفات، وحفظها، وتشغيلها، وبناء التطبيقات.

Explorer

يعرض جميع ملفات المشروع داخل مجلد واحد.

مثال:

```
My Project
|
├─ main.u
├─ style.u
├─ login.u
├─ database.u
└─ images
```

يمكن من خلاله إنشاء ملفات جديدة، أو حذف الملفات، أو إعادة تسميتها.

Editor

هو المكان الذي يكتب فيه المبرمج كود **UNL**.

يوفر العديد من الميزات مثل:

- تلوين الأكواد.
- ترقيم الأسطر.
- طي الكود.
- البحث والاستبدال.
- الإكمال التلقائي.
- تنسيق الكود.

Console

تعرض نافذة **Console** ناتج تشغيل البرنامج.

فعند تنفيذ:

```
Print("Hello UNL")  
  
Unteck run.u(bg)
```

ستظهر النتيجة داخل Console.

كما تعرض رسائل الأخطاء والتحذيرات أثناء التشغيل.

Compiler

يتولى Compiler ترجمة المشروع بالكامل.

عند الضغط على زر **Build** يبدأ تحويل المشروع إلى برنامج قابل للتنفيذ.

AI Assistant

يوفر مساعدًا ذكيًا داخل بيئة التطوير للمساعدة في كتابة الأكواد، وشرح الأخطاء، واقتراح الحلول، وتحسين المشاريع.

إنشاء مشروع جديد

لإنشاء مشروع جديد:

1. افتح قائمة **File**.

2. اختر **New Project**.

3. اكتب اسم المشروع.

4. حدد مكان الحفظ.

5. اضغط **Create**.

سيقوم البرنامج بإنشاء هيكل المشروع تلقائيًا.

فتح مشروع

لفتح مشروع موجود:

File ▾

Open Project ▾

اختيار المجلد ▾

Open ▾

تشغيل المشروع

بعد الانتهاء من كتابة الكود:

اضغط على:

Run ▶

أو استخدم الاختصار الذي توفره بيئة التطوير.

سيبدأ البرنامج بالتجميع ثم التشغيل.

بناء التطبيق

عند الانتهاء من المشروع يمكن إنشاء نسخة قابلة للنشر باستخدام:

Build

ثم اختيار نوع المشروع المطلوب.

مثل:

Desktop ▪

APK ▪

Console ▪

بحسب نوع التطبيق.

مميزات UNL Studio

- واجهة بسيطة.
- سرعة عالية.
- استهلاك منخفض للذاكرة.
- إكمال تلقائي.
- تلوين الأكواد.
- كشف الأخطاء.
- مترجم مدمج.
- دعم المشاريع الكبيرة.
- إدارة الملفات.
- دعم الذكاء الاصطناعي.
- بناء التطبيقات بسهولة.

أفضل طريقة للعمل

يوصى بتنظيم المشروع بالشكل التالي:

```
Project
|
├─ main.u
├─ pages
├─ styles
├─ database
├─ api
├─ assets
└─ modules
```

يسهل هذا التنظيم إدارة المشاريع الكبيرة.

ملخص الفصل

يمثل **UNL Studio** البيئة الرسمية لتطوير تطبيقات لغة **UNL**، حيث يجمع بين محرر الأكواد، والمترجم، وأدوات إدارة المشاريع، والمساعد الذكي، في واجهة واحدة تساعد المطور على بناء التطبيقات بسرعة واحترافية.

الفصل الثاني والثلاثون

أفضل الممارسات (Best Practices)

بعد تعلم لغة **UNL** وأدواتها المختلفة، يبقى العامل الأهم في نجاح أي مشروع هو طريقة كتابة الكود. قد يكتب مبرمجان نفس البرنامج، لكن أحدهما يكتب كودًا منظمًا وسهل التطوير، بينما يكتب الآخر كودًا يصعب فهمه وصيانته.

في هذا الفصل سنتعرف على مجموعة من أفضل الممارسات التي يُنصح باتباعها عند تطوير المشاريع باستخدام لغة **UNL**.

اختر أسماء واضحة

احرص دائمًا على اختيار أسماء تصف وظيفة المتغير أو الدالة أو الكائن.

جيد:

```
userName  
studentScore  
totalPrice
```

غير جيد:

```
a  
x  
abc  
test1
```

قسم المشروع إلى ملفات

لا تضع جميع الأكواد داخل ملف واحد.

استخدم **Link.file** لتقسيم المشروع.

مثال:

```
main.u
login.u
database.u
api.u
style.u
```

يسهل ذلك قراءة المشروع وتطويره.

أنشئ دوال صغيرة

يفضل أن تؤدي كل دالة مهمة واحدة فقط.

بدلاً من إنشاء دالة تحتوي مئات الأسطر، قسمها إلى عدة دوال أصغر.

استخدم التعليقات عند الحاجة

التعليقات تساعد على فهم الكود، لكن لا تكثر منها دون داعٍ.

مثال:

```
// التحقق من بيانات المستخدم
```

استعمل الثوابت

إذا كانت القيمة لن تتغير فاستخدم الثوابت مثل:

▪ `$.pi`:

▪ `$.e`:

▪ `$.tau`:

بدلاً من كتابة الأرقام يدوياً.

تحقق من المدخلات

لا تعتمد على أن المستخدم سيدخل بيانات صحيحة دائمًا.

تحقق من:

- النصوص الفارغة.
- القيم السالبة.
- البريد الإلكتروني.
- كلمات المرور.
- الملفات.

استخدم معالجة الأخطاء

يفضل وضع العمليات التي قد تفشل داخل Try و Catch حتى لا يتوقف البرنامج بالكامل عند حدوث خطأ.

لا تكرر الكود

إذا لاحظت أنك كتبت نفس الكود أكثر من مرة، فحول هذا الجزء إلى دالة أو ملف مستقل.

نظم الكود

ضع فراغات بين الأقسام، واستخدم المسافات والمحاذاة بطريقة ثابتة.

الكود المنظم أسهل في القراءة والمراجعة.

قسم المشاريع الكبيرة

يفضل تقسيم المشروع إلى مجلدات مثل:


```
assets
pages
styles
database
api
models
modules
```

اختبر البرنامج باستمرار

لا تنتظر حتى تنتهي من المشروع بالكامل، بل شغل البرنامج بعد كل جزء جديد للتأكد من أن كل شيء يعمل كما هو متوقع.

اجعل الواجهات بسيطة

في تطبيقات **un.web** و **un.gui** ركز على:

- وضوح الألوان.
- سهولة الاستخدام.
- حجم الخط المناسب.
- ترتيب العناصر.

احم البيانات

عند التعامل مع كلمات المرور استخدم دوال التشفير الموجودة في **un.cyber**، ولا تحفظ كلمات المرور كنص عادي.

استخدم الوحدات المناسبة

لا تضع جميع الوظائف في وسيط واحد.

على سبيل المثال:

▪ **un.net** للشبكات.

▪ **un.ai** للذكاء الاصطناعي.

▪ **un.cyber** للأمان.

▪ **un.gui** للواجهات.

ملخص الفصل

اتباع أفضل الممارسات يجعل مشاريع **UNL** أكثر تنظيماً، وأسهل في التطوير، وأسرع في الصيانة، ويقلل من الأخطاء مع نمو المشروع.

الفصل الثالث والثلاثون

الأخطاء الشائعة وكيفية تجنبها

يرتكب جميع المبرمجين أخطاء أثناء البرمجة، سواء كانوا مبتدئين أو محترفين. معرفة هذه الأخطاء مسبقًا تساعد على توفير الكثير من الوقت والجهد.

نسيان نهاية البرنامج

كل برنامج في **UNL** يجب أن ينتهي بـ:

```
Unteck run.u(bg)
```

نسيان هذا السطر يؤدي إلى عدم اكتمال تنفيذ البرنامج.

كتابة اسم متغير بشكل مختلف

لغة **UNL** غير حساسة لحالة الأحرف، لكن تغيير اسم المتغير نفسه يؤدي إلى خطأ.

مثال:

```
userName
```

ليس هو نفسه:

```
user_Name
```

نسيان إرجاع القيمة

إذا كانت الدالة يجب أن تعيد قيمة، فتأكد من استخدام:

```
return
```

في المكان المناسب.

استخدام متغير قبل تعريفه

يجب إنشاء المتغير أولاً ثم استخدامه.

نسيان ربط الملفات

إذا كان المشروع يعتمد على عدة ملفات، فتأكد من ربطها باستخدام:

```
Link.file(...)
```

عدم معالجة الأخطاء

قد تتوقف بعض العمليات مثل الشبكات أو قواعد البيانات، لذلك يفضل دائماً استخدام Try و Catch.

استخدام الوسيط الخطأ

كل مجال له وسيطه الخاص.

أمثلة:

▪ **un.web** لتطوير الويب.

▪ **un.back** للخوادم.

▪ **un.ai** للذكاء الاصطناعي.

▪ **un.net** للشبكات.

▪ **un.gui** لواجهات سطح المكتب.

عدم تنظيم المشروع

وضع جميع الملفات داخل مجلد واحد يجعل المشروع صعب الإدارة.

يفضل تقسيم الملفات حسب نوعها.

إهمال التعليقات

في المشاريع الكبيرة، قد يصعب فهم الكود بعد أشهر من كتابته.

إضافة تعليقات مختصرة في الأماكن المهمة تساعد على صيانة المشروع مستقبلاً.

تكرار الكود

تكرار نفس التعليمات عدة مرات يزيد حجم المشروع ويجعل تعديله أصعب.

الحل هو إنشاء دالة أو وحدة يمكن إعادة استخدامها.

تجاهل التحقق من البيانات

قبل معالجة أي بيانات، تحقق من صحتها.

مثال:

- هل البريد الإلكتروني صحيح؟
- هل كلمة المرور قوية؟
- هل الملف موجود؟
- هل الرقم ضمن المجال المطلوب؟

عدم اختبار المشروع

اختبر كل جزء مباشرة بعد الانتهاء منه، ولا تؤجل الاختبار إلى نهاية المشروع.

نصائح للمبتدئين

- ابدأ بمشاريع صغيرة.
- اقرأ رسائل الخطأ بعناية.
- لا تخف من التجربة.
- قسم المشكلة إلى أجزاء صغيرة.
- احتفظ بنسخة احتياطية من المشروع.

ملخص الفصل

الأخطاء جزء طبيعي من عملية البرمجة، لكن التعرف عليها مبكرًا واتباع أساليب صحيحة في كتابة الكود يساعد على بناء مشاريع أكثر استقرارًا وجودة باستخدام لغة **UNL**.

الفصل الرابع والثلاثون

مشاريع كاملة من الصفر حتى الاحتراف

بعد تعلم أساسيات لغة UNL، يأتي الوقت لتطبيق ما تعلمته في مشاريع حقيقية. لا تجعل التعلم مقتصرًا على قراءة الأمثلة، بل حاول تنفيذ مشاريع متكاملة، فكل مشروع سيضيف خبرة جديدة ويكشف أفكارًا لم تكن لتلاحظها أثناء قراءة الشرح فقط. في هذا الفصل ستجد مجموعة من المشاريع المقترحة، مرتبة من السهل إلى المتقدم.

المشروع الأول: آلة حاسبة

المستوى: مبتدئ

ستتعلم في هذا المشروع:

- المتغيرات.
- العمليات الحسابية.
- إدخال البيانات.
- طباعة النتائج.
- الشروط.
- الدوال.

الميزات المقترحة:

- الجمع.
- الطرح.
- الضرب.
- القسمة.
- الأس.
- الجذر التربيعي.
- حساب النسبة المئوية.

بعد إنهاء المشروع ستكون قادرًا على بناء برامج Console بسيطة.

المشروع الثاني: تطبيق ملاحظات

المستوى: مبتدئ، إلى متوسط

المفاهيم المستخدمة:

- الكائنات.
- قواعد البيانات.
- حفظ الملفات.
- الواجهات.
- البحث.

الميزات:

- إنشاء ملاحظة.
- تعديلها.
- حذفها.
- البحث داخل الملاحظات.
- تصنيف الملاحظات.

المشروع الثالث: موقع شخصي

المستوى: متوسط

الوسائط المستخدمة:

- un.web
- Style

الميزات:

- الصفحة الرئيسية.
- صفحة التواصل.
- معرض الصور.
- الوضع الليلي.
- التصميم المتجاوب.

المشروع الرابع: واجهة سطح مكتب

المستوى: متوسط

الوسيط المستخدم:

▪ un.gui

الميزات:

▪ نافذة رئيسية.

▪ أزرار.

▪ قوائم.

▪ مربعات نص.

▪ أيقونات.

▪ رسائل تنبيه.

المشروع الخامس: نظام تسجيل دخول

المستوى: متوسط

الوسائط المستخدمة:

▪ un.back

▪ un.cyber

▪ Database

المميزات:

▪ إنشاء حساب.

▪ تسجيل الدخول.

▪ تشفير كلمة المرور.

▪ إدارة الجلسات.

▪ JWT.

▪ صلاحيات المستخدم.

المشروع السادس: متجر إلكتروني

المستوى: متقدم

المكونات:

- واجهة أمامية.
- واجهة خلفية.
- قاعدة بيانات.
- واجهة إدارة.
- نظام دفع.
- إدارة المنتجات.
- إدارة المستخدمين.

المشروع السابع: مساعد ذكاء اصطناعي

المستوى: متقدم

الوسيط المستخدم:

un.ai ▪

الميزات:

- استقبال الأسئلة.
- إرسالها إلى نموذج الذكاء الاصطناعي.
- عرض الإجابة.
- حفظ سجل المحادثة.
- تغيير النموذج.
- التحكم في Temperature.

المشروع الثامن: لوحة تحكم API

المستوى: متقدم

الوسيط المستخدم:

un.net ▪

الميزات:

▪ إرسال طلبات GET.

▪ إرسال POST.

▪ رفع الملفات.

▪ تنزيل الملفات.

▪ WebSocket.

▪ Headers.

▪ عرض النتائج.

المشروع التاسع: نظام إنترنت الأشياء

المستوى: احترافي

الوسيط المستخدم:

un.iot ▪

المشروع يتضمن:

▪ الاتصال بجهاز ESP32.

▪ تشغيل وإطفاء الأجهزة.

▪ قراءة المستشعرات.

▪ مراقبة الحرارة والرطوبة.

▪ استقبال البيانات مباشرة.

▪ إنشاء أحداث تلقائية.

المشروع العاشر: تطبيق APK متكامل

المستوى: احترافي

الوسائط المستخدمة:

- un.apk
- un.ai
- un.net
- Database

المميزات:

- شاشة تسجيل الدخول.
- صفحات متعددة.
- اتصال بالإنترنت.
- قاعدة بيانات.
- ذكاء اصطناعي.
- تحديثات التطبيق.
- أيقونة مخصصة.
- شاشة إعدادات.

ماذا بعد؟

بعد إنهاء هذه المشاريع ستكون قد استخدمت معظم إمكانيات لغة UNL، وستصبح قادرًا على تطوير تطبيقات حقيقية لمختلف المنصات، بدءًا من برامج Console البسيطة وحتى تطبيقات سطح المكتب والويب والهواتف والخوادم.

الفصل الخامس والثلاثون

الخاتمة

وصلت الآن إلى نهاية هذا الكتاب، لكنها ليست نهاية الرحلة، بل بدايتها.

لقد تعرفت على لغة UNL، وفهمت فلسفتها، وتعلمت أساسياتها، ثم انتقلت إلى الدوال، والكائنات، والمجموعات، ومعالجة الأخطاء، والبرمجة غير المتزامنة، وقواعد البيانات، والواجهات، والشبكات، والذكاء الاصطناعي، والأمن السيبراني، وإنترنت الأشياء، وبناء المشاريع المتكاملة.

الهدف من هذا الكتاب لم يكن تعليم أوامر اللغة فقط، بل تعليم طريقة التفكير التي تساعد على بناء برامج منظمة وقابلة للتطوير.

لغة UNL هي مشروع فردي بدأه **خالف إبراهيم خليل** بدافع الشغف بالتقنية وصناعة الأدوات البرمجية. تطورت الفكرة خطوة بعد خطوة حتى أصبحت مشروعًا يطمح إلى توفير لغة حديثة تجمع بين سهولة الاستخدام وتعدد المجالات في بيئة واحدة.

إذا وصلت إلى هذه الصفحة، فهذا يعني أنك استثمرت وقتًا وجهودًا في التعلم، وهذه هي أهم خطوة في طريق أي مبرمج ناجح. لا تتوقف عند الأمثلة الموجودة في الكتاب، بل حاول تطوير أفكارك الخاصة، واصنع مشاريع جديدة، وشارك أعمالك مع الآخرين، واستمر في التعلم والتجربة، فكل مشروع جديد سيضيف إليك خبرة جديدة.

وفي الختام، أتمنى أن يكون هذا الكتاب قد ساعدك على فهم لغة UNL، وأن يكون بداية لمشاريع مميزة تبنيها بنفسك في المستقبل.

شكرًا لكل من قرأ هذا الكتاب حتى النهاية، ولكل من يستخدم لغة UNL لصناعة فكرة جديدة أو حل مشكلة أو تعلم مهارة جديدة.

أتمنى أن تجد ثمرة الجهد المبذول في هذا المشروع، وأن يكون هذا العمل قد قدم فائدة حقيقية لكل من قرأه.

بالتوفيق، وإلى لقاء في إصدار جديد من لغة UNL وكتابها.